

Machine Learning Formula & Decision Sheet

machine-learning-formula-decision-sheet

Contents

Part I — Supervised Learning	2
1 Learning Notation and Prediction Setup	2
2 Linear Regression	4
3 Cost Functions and Gradient Descent	5
4 Multiple and Polynomial Regression, Vectorization, and Feature Scaling	8
5 Logistic Regression and Decision Boundaries	10
6 Logistic Loss and Classification Decisions	12
7 Overfitting and Regularization	14
8 Theory Checkpoint I	17
Part II — Neural Networks, Model Diagnosis, and Tree Ensembles	18
9 Neural Networks and Forward Propagation	18
10 Activations, Multiclass Classification, and Softmax	21
11 Neural Network Training, Backpropagation, and Adam	24
12 Evaluation, Bias/Variance, and Learning Curves	27
13 ML Strategy and Error Analysis	30
14 Decision Trees, Entropy, and Information Gain	36
15 Tree Ensembles and Boosting	39
16 Theory Checkpoint II	44
Part III — Clustering, Recommendations, and Reinforcement Learning	45
17 K-means Clustering	45
18 Anomaly Detection	48
19 Collaborative Filtering	50
20 Content-Based Recommender Systems	53
21 Principal Component Analysis	54
22 Reinforcement Learning Foundations	57

Part I — Supervised Learning

1 Learning Notation and Prediction Setup

Notation and Symbols

Symbol	Meaning in this chapter
m	Number of training examples.
n	Number of input features available for each example.
$x^{(i)} \in \mathbb{R}^n$	Feature vector for example i .
$y^{(i)}$	Observed target: a number for regression or a class label for classification.
$f_\theta(x)$	Prediction function parameterized by θ .
$\hat{y}$	Model prediction $f_\theta(x)$, distinguished from the observed target y .
$J(\theta)$	Average cost used to select parameters.

1.1 A Learning Problem as Three Objects

Data, model, and objective

A supervised learning problem is defined only after three choices are explicit:

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^m, \quad \hat{y}^{(i)} = f_\theta(x^{(i)}), \quad \theta^* = \arg \min_{\theta} J(\theta).$$

Use: Keep the observed data, prediction rule, and optimization target separate. Two models can receive the same data yet solve different problems because their costs or outputs differ.

Notes: A parameter is learned from data. A hyperparameter, such as a regularization strength or decision threshold, is selected outside the direct fitting step.

Regression versus classification

Task	Model output	Typical question
Regression	$\hat{y} \in \mathbb{R}$	How much, how long, or how many?
Binary classification	$\hat{p} = P(y = 1 x)$ and a class decision	Does an event occur?
Multiclass classification	$(\hat{p}_1, \dots, \hat{p}_K)$	Which mutually exclusive class?

1.2 Training, Validation, and Generalization

Empirical cost

The training cost averages losses computed on examples available during fitting:

$$J_{\text{train}}(\theta) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(f_\theta(x^{(i)}), y^{(i)}).$$

Derivation: The average gives each training observation one unit of influence before weights or resampling are introduced. Minimizing this expression chooses a model that fits observed examples; it does not by itself certify performance on unseen examples.

Use: Define precisely what training optimizes before reasoning about model quality.

Generalization gap

Let J_{val} measure loss on held-out validation examples. The gap

$$J_{\text{val}}(\theta^*) - J_{\text{train}}(\theta^*)$$

is evidence about generalization, not an alternative training objective unless a separate tuning procedure uses validation results. *Use:* Distinguish fitting error from the evidence used to choose complexity, features, thresholds, or regularization.

Notes: Repeatedly optimizing design choices against one validation set can itself overfit that set. A final test set is retained for an unbiased final estimate.

1.3 Data Matrix Convention

Examples by features

For compact vectorized formulas in this sheet, rows represent examples:

$$X = \begin{bmatrix} (x^{(1)})^T \\ (x^{(2)})^T \\ \vdots \\ (x^{(m)})^T \end{bmatrix} \in \mathbb{R}^{m \times n}, \quad y = \begin{bmatrix} y^{(1)} & \cdots & y^{(m)} \end{bmatrix}^T.$$

Use: Read Xw as one predicted linear score for every training example.

Notes: The neural-network chapter will explicitly state its batch convention again; the important rule is internal consistency, not one universal orientation.

1.4 Population Risk and Sampling Error

What training cost estimates

Definition. A training set is a finite sample from a target population. The quantity ultimately desired is the expected risk

$$R(\theta) = \mathbb{E}_{(X,Y) \sim P_{\text{target}}} [\mathcal{L}(f_{\theta}(X), Y)], \quad \hat{R}_{\text{train}}(\theta) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(f_{\theta}(x^{(i)}), y^{(i)}).$$

Derivation: Empirical risk replaces an unavailable population expectation by an average over observed examples. Minimizing it selects $\hat{\theta} = \arg \min_{\theta} \hat{R}_{\text{train}}(\theta)$, but the selection itself can exploit accidental patterns in this sample.

Use: Keep the distinction between optimizing observed examples and measuring behavior on future examples.

Notes: Large training sets narrow sampling variation only when their distribution represents the target use case. More examples from the wrong population do not remove deployment mismatch.

Risk decomposition used for decisions

For any trained model, compare three conceptual quantities:

$$\underbrace{R_{\text{baseline}}}_{\text{irreducible or acceptable reference}}, \quad \underbrace{\hat{R}_{\text{train}}(\hat{\theta})}_{\text{fit}}, \quad \underbrace{\hat{R}_{\text{val}}(\hat{\theta})}_{\text{generalization estimate}}.$$

The train-to-baseline gap suggests insufficient fit or high avoidable bias; the validation-to-training gap suggests variance or mismatch. *Use:* Choose whether to alter representation/model capacity or gather data and regularize, rather than choosing improvements from intuition alone.

1.5 Data Split Discipline

Selection changes what remains unbiased

Let λ denote any design choice: degree, features, architecture, regularization, or threshold. Then

$$\lambda^* = \arg \min_{\lambda \in \Lambda} J_{\text{val}}(\hat{\theta}_{\lambda}), \quad J_{\text{test}}(\hat{\theta}_{\lambda^*}) \text{ is evaluated once after selection.}$$

Derivation: Once the validation results influence λ^* , the validation set has become part of the design process. A separate test set is needed to estimate the selected system without that feedback.

Cue	Reading
Training set	Fits numeric parameters such as weights, centroids, or embeddings.
Validation set	Chooses models, thresholds, features, and intervention priorities.
Test set	Estimates final selected-system performance; do not repeatedly tune to it.
Deployment monitoring	Detects later drift, operational cost, or constraint failures.

Notes: Data leakage includes preprocessing fitted on validation or test examples. Scaling means, vocabularies, imputation rules, and feature-selection decisions are learned only from training data and then applied unchanged.

1.6 From Prediction to Decision

A score is not yet an action

A prediction can be a number, a probability, a rank, or an action value. A business or scientific action adds a rule d :

$$x \mapsto f_{\theta}(x) \mapsto d(f_{\theta}(x)) \in \mathcal{A}.$$

Use: Separate model-quality questions from operating-policy questions. A good probability model can be deployed with a poor threshold; a useful ranking model can be deployed with a poor capacity constraint.

Notes: Later chapters repeatedly apply this separation to classification thresholds, anomaly alarms, recommendation ranking, and reinforcement-learning policies.

2 Linear Regression

Notation and Symbols

Symbol	Meaning in this chapter
$x^{(i)}$	Features describing example i .
$y^{(i)} \in \mathbb{R}$	Continuous observed target.
$w \in \mathbb{R}^n, b \in \mathbb{R}$	Slope/feature weights and intercept.
$f_{w,b}(x)$	Linear prediction $w^T x + b$.
$e^{(i)}$	Residual $\hat{y}^{(i)} - y^{(i)}$.
$J(w, b)$	Squared-error cost for fitting the regression model.

2.1 Prediction Rule

Single and multiple feature models

For one feature the prediction is a line; for multiple features it is a hyperplane:

$$f_{w,b}(x) = wx + b \quad (n = 1), \quad f_{w,b}(x) = w^T x + b = \sum_{j=1}^n w_j x_j + b.$$

Use: Predict a numeric outcome when a linear combination of measured features is a reasonable first approximation.

Notes: A linear model can still express curved relationships after engineered features such as x_1^2 or $x_1 x_2$ are introduced. It is linear in its parameters, not necessarily in the raw measurements.

2.2 Squared Error Objective

Per-example loss and average cost

$$\mathcal{L}^{(i)}(w, b) = \frac{1}{2} \left(f_{w,b}(x^{(i)}) - y^{(i)} \right)^2, \quad J(w, b) = \frac{1}{2m} \sum_{i=1}^m \left(f_{w,b}(x^{(i)}) - y^{(i)} \right)^2.$$

Derivation: Squaring makes positive and negative residuals contribute rather than cancel. The factor 1/2 does not alter the minimizer; it cancels the factor 2 created when differentiating the square.

Use: Fit a regression function by making large prediction errors disproportionately expensive.

Notes: Squared error corresponds to a Gaussian-noise likelihood assumption when residual variance is constant. It is sensitive to severe outliers.

Vectorized form

With $\hat{y} = Xw + b\mathbf{1}$ and $e = \hat{y} - y$,

$$J(w, b) = \frac{1}{2m} \|Xw + b\mathbf{1} - y\|_2^2.$$

This expression exposes the geometry: fitting chooses a point in the column space of the augmented feature matrix that lies closest to the observed target vector.

2.3 How Parameters Affect Predictions

Cue	Reading
b changes	A common vertical shift for all predictions; it changes the baseline outcome.
w_j changes	The prediction response to one unit of feature j , holding represented features fixed.
Feature scale changes	The numeric size of w_j , not necessarily the real-world importance of feature j .
Residual pattern remains curved	A linear basis may be inadequate even if optimization converges.

Model limits

The model estimates association inside its feature representation. A fitted coefficient does not become a causal effect merely because optimization found a small cost. Unmeasured confounding, selection bias, and extrapolation outside the training range can all invalidate a causal reading.

2.4 Least Squares as an Optimization Problem

Augmented matrix convention

Absorb the intercept into a column of ones:

$$\tilde{X} = \begin{bmatrix} 1 & (x^{(1)})^T \\ \vdots & \vdots \\ 1 & (x^{(m)})^T \end{bmatrix} \in \mathbb{R}^{m \times (n+1)}, \quad \tilde{w} = \begin{bmatrix} b \\ w \end{bmatrix}, \quad J(\tilde{w}) = \frac{1}{2m} \|\tilde{X}\tilde{w} - y\|_2^2.$$

Derivation: Writing the intercept as an additional coefficient lets all first-order conditions be stated with one matrix derivative.

Normal equation

When $\tilde{X}^T \tilde{X}$ is invertible, setting the gradient to zero gives

$$\nabla_{\tilde{w}} J = \frac{1}{m} \tilde{X}^T (\tilde{X}\tilde{w} - y) = 0, \quad \tilde{w}^* = (\tilde{X}^T \tilde{X})^{-1} \tilde{X}^T y.$$

Derivation: The optimum makes the residual vector orthogonal to every represented feature direction: $\tilde{X}^T (y - \tilde{X}\tilde{w}^*) = 0$. Thus least squares is projection of y onto the column space of $\tilde{X}$.

Use: Understand the mathematical solution and diagnose feature redundancy; for large feature sets, numerical solvers or gradient methods are preferred to forming an explicit inverse.

Notes: If columns are dependent, the solution is not unique. A pseudoinverse gives a minimum-norm solution; regularization makes the optimization well conditioned.

3 Cost Functions and Gradient Descent

Notation and Symbols

Symbol	Meaning in this chapter
$\alpha > 0$	Learning rate controlling one update step.
$J(w, b)$	Objective minimized by training.
$\frac{\partial J}{\partial w_j}$	Slope of the cost with respect to feature weight j .
$\nabla_w J$	Vector of all weight derivatives.
t	Optimization iteration index.
$e^{(i)}$	Regression residual $f_{w,b}(x^{(i)}) - y^{(i)}$.

3.1 Gradient Descent Update

Simultaneous parameter update

$$w_j \leftarrow w_j - \alpha \frac{\partial J(w, b)}{\partial w_j}, \quad b \leftarrow b - \alpha \frac{\partial J(w, b)}{\partial b}.$$

Derivation: A derivative is the local direction of increase in cost. Subtracting a positive multiple of it steps locally downhill. All derivatives must be computed from the old parameter values before any parameter is replaced.

Use: Iteratively reduce an objective when closed-form fitting is inconvenient or when the same optimizer will generalize to richer models.

3.2 Regression Gradients

Derivative with respect to the intercept

Starting from

$$J(w, b) = \frac{1}{2m} \sum_i \left(w^T x^{(i)} + b - y^{(i)} \right)^2,$$

the chain rule gives

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^m \left(f_{w,b}(x^{(i)}) - y^{(i)} \right) = \frac{1}{m} \sum_i e^{(i)}.$$

Derivation: Increasing b increases every prediction by one unit, so the average signed residual tells whether the fitted line is collectively too high or too low.

Derivative with respect to a feature weight

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m \left(f_{w,b}(x^{(i)}) - y^{(i)} \right) x_j^{(i)}.$$

Derivation: The residual is weighted by feature x_j because increasing w_j affects high- x_j examples more strongly. At a stationary optimum, residuals are uncorrelated with each included feature in this training-sample sense.

Vectorized gradient

$$\nabla_w J = \frac{1}{m} X^T (Xw + b\mathbf{1} - y), \quad \frac{\partial J}{\partial b} = \frac{1}{m} \mathbf{1}^T (Xw + b\mathbf{1} - y).$$

Use: Implement all parameter gradients without explicit loops over features.

3.3 Reading an Optimization Trace

Cue	Reading
<i>J decreases steadily</i>	Learning rate is at least stable; continue until change is negligible for the purpose.
<i>J increases or oscillates</i>	The step is too aggressive or implementation is incorrect; reduce α after checking formulas.
<i>J decreases very slowly</i>	Scaling or a larger stable α may matter more than changing the model family.
<i>Small training J</i>	Optimization fit the training data; it says nothing alone about validation performance.

Convergence for linear regression

For squared-error linear regression, $J(w, b)$ is convex. Any global minimum has the same minimum objective value, although redundant features can allow more than one parameter vector to represent equivalent predictions. *Notes:* Convexity avoids poor local minima; it does not avoid poor generalization, outliers, missing variables, or an inappropriate target.

3.4 Why the Update Can Decrease Cost

First-order descent reasoning

For a small displacement $\Delta\theta$, a differentiable objective obeys

$$J(\theta + \Delta\theta) \approx J(\theta) + \nabla J(\theta)^T \Delta\theta.$$

Choosing $\Delta\theta = -\alpha \nabla J(\theta)$ yields

$$J(\theta - \alpha \nabla J(\theta)) \approx J(\theta) - \alpha \|\nabla J(\theta)\|_2^2.$$

Derivation: The negative gradient gives the steepest local decrease among equal-length directions. The approximation only guarantees useful motion when α is small enough relative to curvature.

Use: Interpret gradient descent as a consequence of the objective, rather than a memorized update rule.

3.5 Reading the Learning Curve of the Optimizer

Cost trace diagnostics

Cue	Reading
J decreases steadily but very slowly	Increase α cautiously, scale features, or improve vectorization.
J alternates or increases	Reduce α ; check gradients and overflow before extending training.
J becomes NaN or enormous	Suspect excessive step size, extreme feature scale, or numerical errors.
J_{train} converges but validation is poor	This is a generalization problem, not an optimizer convergence problem.

Convergence test

A practical stopping criterion compares a recent improvement with a tolerance:

$$\left| J(\theta^{(t-1)}) - J(\theta^{(t)}) \right| < \varepsilon_J, \quad \text{or} \quad \|\nabla J(\theta^{(t)})\|_2 < \varepsilon_g.$$

Notes: A stopped optimizer has found a numerically stable point for the stated objective; it has not demonstrated that the objective encodes the right task.

3.6 Batch, Stochastic, and Mini-Batch Objectives

Noisy gradient estimates

Full-batch descent uses

$$g = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} \mathcal{L}^{(i)}.$$

For mini-batch B_t of size $|B_t|$,

$$g_t = \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_{\theta} \mathcal{L}^{(i)}, \quad \theta_{t+1} = \theta_t - \alpha g_t.$$

Derivation: When a batch is sampled representatively, $\mathbb{E}[g_t] = g$; updates are unbiased but noisy. Smaller batches trade cheaper steps and useful stochasticity for a less smooth cost trace.

Use: Connect the exact linear-regression update with the mini-batch training used for neural networks later in the sheet.

4 Multiple and Polynomial Regression, Vectorization, and Feature Scaling

Notation and Symbols

Symbol	Meaning in this chapter
x_j	Raw feature j , such as size or age.
$\phi(x)$	Engineered feature map, for example $[x, x^2, x^3]^T$.
μ_j, σ_j	Feature mean and scale measure used in normalization.
z_j	Scaled feature $(x_j - \mu_j)/\sigma_j$.
X	Matrix whose rows are examples and columns are represented features.
w	Weight vector compatible with the chosen feature map.

4.1 Multiple Regression

Many features in one score

$$f_{w,b}(x) = w^T x + b = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b.$$

Use: Combine distinct measurements, such as property size, room count, and age, into a single numeric prediction.

Notes: Coefficients are interpreted relative to the feature representation and the data distribution. Highly correlated features can make individual coefficients unstable while predictions remain adequate.

4.2 Feature Engineering and Polynomial Regression

Linear fitting in a nonlinear basis

$$\phi(x) = \begin{bmatrix} x \\ x^2 \\ x^3 \end{bmatrix}, \quad f_{w,b}(x) = w^T \phi(x) + b = w_1 x + w_2 x^2 + w_3 x^3 + b.$$

Derivation: Gradient descent still differentiates a linear combination of represented features. Nonlinear curvature arises from the basis $\phi(x)$, while the model is linear in learned weights w .

Use: Represent smooth curvature or interactions without changing the fundamental linear-regression training mechanism.

Notes: Higher-degree features enlarge the hypothesis space and can increase variance; validation performance and regularization become decisive.

Interaction term

An interaction lets the contribution of one feature depend on another:

$$\phi(x_1, x_2) = \begin{bmatrix} x_1 & x_2 & x_1 x_2 \end{bmatrix}^T.$$

For example, floor area and location quality may jointly contribute more than their separate additive terms imply.

4.3 Vectorization

Predictions and gradient for all examples

$$\hat{y} = Xw + b\mathbf{1}, \quad e = \hat{y} - y, \quad \nabla_w J = \frac{1}{m} X^T e.$$

Derivation: Matrix multiplication performs the same dot product for every example and then aggregates feature-weighted residuals. It is mathematically identical to loops but enables optimized linear algebra execution.

Use: Express implementation and algebra compactly, and reduce opportunities for indexing errors.

4.4 Feature Scaling

Mean normalization or standardization

$$z_j = \frac{x_j - \mu_j}{\sigma_j}, \quad \mu_j = \frac{1}{m} \sum_i x_j^{(i)}.$$

Here σ_j may be a standard deviation or a range chosen consistently. *Derivation:* When one feature varies over thousands and another over tenths, equal changes in their parameters cause very different prediction moves. Scaling makes contours of J less elongated, so one learning rate can make useful progress in each coordinate.

Use: Improve gradient-descent conditioning; apply training-set scaling parameters to validation, test, and future inputs.

Notes: Scaling changes numerical conditioning and coefficient units, not the underlying information. It must not be fitted on held-out data.

4.5 Representation Determines What “Linear” Means

Basis expansion

Define a feature map $\phi : \mathbb{R}^n \rightarrow \mathbb{R}^q$. Regression remains linear in its parameters although it can be nonlinear in raw measurements:

$$f_{w,b}(x) = w^T \phi(x) + b.$$

For two original features, a quadratic map may be

$$\phi(x_1, x_2) = (x_1, x_2, x_1^2, x_1 x_2, x_2^2)^T.$$

Derivation: Training adjusts coefficients multiplying fixed basis functions. It does not learn which nonlinear transformation should exist; feature design chooses the hypothesis family before fitting.

Use: Build curved trends and interactions while retaining interpretable linear optimization.

Notes: Polynomial degree increases capacity rapidly. With n original features, all monomials through degree d can be numerous, increasing variance and making regularization more important.

4.6 Scaling and Conditioning

Standardization

For training-derived mean μ_j and standard deviation $s_j > 0$,

$$z_j = \frac{x_j - \mu_j}{s_j}, \quad \mu_j = \frac{1}{m} \sum_i x_j^{(i)}, \quad s_j^2 = \frac{1}{m} \sum_i (x_j^{(i)} - \mu_j)^2.$$

Apply the same μ_j, s_j to validation, test, and deployment examples. *Derivation:* Changing coordinates does not remove represented predictive relationships. It rounds elliptical cost contours toward circles, allowing one learning rate to make useful progress in all weight directions.

Converting coefficients back to raw units

If the scaled model is $\hat{y} = b_z + \sum_j w_{z,j} z_j$, then

$$\hat{y} = \left(b_z - \sum_j \frac{w_{z,j} \mu_j}{s_j} \right) + \sum_j \frac{w_{z,j}}{s_j} x_j.$$

Use: Train efficiently in standardized coordinates while interpreting effects in the original units.

4.7 Feature Engineering Decision Rules

Cue	Reading
Response rises with area but dimensions are supplied	Add $x_{\text{length}}x_{\text{width}}$ as an interaction.
Residual trend curves smoothly	Try polynomial or transformed features and compare validation performance.
High-degree model fits training nearly perfectly	Treat flexibility as a variance risk; tune degree and regularization together.
Future scale or unit differs from training	Preserve transformation units and monitor range drift; scaling alone does not solve shift.

Notes: Vectorization is an implementation improvement; feature engineering is a modeling choice. A fast computation of an unsuitable representation remains an unsuitable model.

5 Logistic Regression and Decision Boundaries

Notation and Symbols

Symbol	Meaning in this chapter
$y \in \{0, 1\}$	Binary class label.
$z = w^T x + b$	Linear logit or score.
$\sigma(z)$	Sigmoid function $1/(1 + e^{-z})$.
$\hat{p}$	Estimated probability $P(y = 1 x)$.
τ	Decision threshold applied to $\hat{p}$.
$\mathcal{B}$	Decision boundary where the selected class switches.

5.1 Probability Model

Sigmoid transformation

$$z = w^T x + b, \quad \hat{p} = f_{w,b}(x) = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Derivation: The sigmoid maps any real score into $(0, 1)$, and its odds satisfy

$$\frac{\hat{p}}{1 - \hat{p}} = e^z, \quad \log \frac{\hat{p}}{1 - \hat{p}} = w^T x + b.$$

Thus logistic regression models the log-odds as linear in represented features.

Use: Estimate a binary-event probability and later convert it into an operational decision using an appropriate threshold.

Notes: A probability estimate and a binary action are not the same object. Costs, capacity, regulation, or safety determine the action threshold.

5.2 Decision Boundaries

Default threshold

With the conventional threshold $\tau = 0.5$,

$$\hat{y} = \begin{cases} 1, & \hat{p} \geq 0.5, \\ 0, & \hat{p} < 0.5, \end{cases} \iff w^T x + b \geq 0.$$

Therefore the boundary is

$$\mathcal{B} = \{x : w^T x + b = 0\}.$$

Derivation: Because $\sigma(0) = 0.5$ and sigmoid is monotonic, thresholding the probability at 0.5 is exactly thresholding its linear logit at zero.

Changing the operating threshold

For any $\tau \in (0, 1)$,

$$\hat{p} \geq \tau \iff w^T x + b \geq \log \frac{\tau}{1 - \tau}.$$

Use: Lower τ when missing positives is especially costly; raise it when false alarms are especially costly.

Notes: Changing a threshold changes classifications without retraining probability parameters; retraining may still be necessary when probability quality or features are inadequate.

5.3 Nonlinear Boundaries Through Features

Polynomial representation

$$z = w_1 x_1 + w_2 x_2 + w_3 x_1^2 + w_4 x_1 x_2 + w_5 x_2^2 + b.$$

The equation $z = 0$ can now describe curves, while training remains logistic regression on engineered features. *Use:* Capture nonlinear separation with controlled feature construction.

Notes: More flexible boundaries can fit noise; use held-out evaluation and regularization rather than judging by training accuracy.

5.4 Log-Odds View of Logistic Regression

Linear score, nonlinear probability

Let $z = w^T x + b$ and $p(y = 1 | x) = \sigma(z)$. Because

$$\frac{\sigma(z)}{1 - \sigma(z)} = e^z,$$

the log-odds are linear:

$$\log \frac{p(y = 1 | x)}{1 - p(y = 1 | x)} = w^T x + b.$$

Derivation: Logistic regression does not claim that probability grows linearly. It claims that one unit of a feature adds w_j to the log-odds when other represented features are held fixed.

Use: Interpret coefficient direction and understand why crossing a probability threshold becomes a linear boundary in represented feature space.

Notes: Coefficient magnitude is sensitive to feature scale. A coefficient cannot be interpreted causally merely because it shifts fitted log-odds.

General threshold boundary

Classify positive when $\hat{p} \geq \tau$, where $0 < \tau < 1$. Then

$$\sigma(w^T x + b) \geq \tau \iff w^T x + b \geq \log \frac{\tau}{1 - \tau}.$$

Derivation: The usual 0.5 threshold has zero right-hand side. Raising τ moves the boundary toward evidence stronger than even odds; lowering it widens the region sent for positive action.

Use: Derive rather than guess how an operating threshold changes a decision boundary.

5.5 Cost-Sensitive Decisions

Optimal threshold under asymmetric action cost

Suppose acting positive on a negative example incurs cost C_{FP} , and acting negative on a positive example incurs C_{FN} . Given calibrated probability p , compare expected costs:

$$\text{Cost}(\hat{y} = 1) = C_{FP}(1 - p), \quad \text{Cost}(\hat{y} = 0) = C_{FN}p.$$

Choose positive when

$$p \geq \tau^* = \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

Derivation: When missing a positive case is expensive, C_{FN} is large and τ^* decreases; more examples are sent to the positive intervention.

Cue	Reading
Medical screen before a confirmatory test	Missing illness may cost more than a follow-up; favor recall with a lower threshold.
Automatic irreversible denial	A false positive action can be severe; demand a higher threshold or human review.
Alert queue with fixed capacity	Select a threshold or top- k policy against capacity and measured yield.
Uncalibrated probability scores	Tune operating points empirically; the cost formula requires probabilities with reliable meaning.

5.6 Represented Nonlinear Boundaries

Boundary is linear only in the chosen features

With $\phi(x) = (x_1, x_2, x_1^2, x_1x_2, x_2^2)$,

$$w^T \phi(x) + b = 0$$

can represent curves in (x_1, x_2) -space. Logistic loss remains convex in w, b for fixed features even though the visible boundary is nonlinear. *Notes:* A highly flexible feature map may separate training examples using noise. Regularization and held-out threshold evaluation should accompany nonlinear boundary design.

6 Logistic Loss and Classification Decisions

Notation and Symbols

Symbol	Meaning in this chapter
$\hat{p}^{(i)}$	Predicted probability for positive class on example i .
$y^{(i)} \in \{0, 1\}$	Observed binary label.
$\mathcal{L}(\hat{p}, y)$	Binary cross-entropy loss.
$J(w, b)$	Average classification cost.
τ	Threshold converting probabilities to actions.
TP, FP, FN, TN	Entries of a binary confusion matrix.

6.1 Cross-Entropy Loss

Single-example loss

$$\mathcal{L}(\hat{p}, y) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p}).$$

Equivalently,

$$\mathcal{L}(\hat{p}, 1) = -\log \hat{p}, \quad \mathcal{L}(\hat{p}, 0) = -\log(1 - \hat{p}).$$

Derivation: If $y = 1$, assigning probability near zero incurs an unbounded penalty; if $y = 0$, assigning probability near one does likewise. A confidently wrong probability is worse than an uncertain one, which squared classification accuracy cannot express.

Use: Train a probabilistic binary classifier and penalize unjustified confidence.

Average logistic cost

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log \hat{p}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right].$$

Derivation: This is negative log-likelihood for independent Bernoulli targets. Minimizing it maximizes the likelihood that the model assigns to the observed labels.

6.2 Gradient of Logistic Regression

Let $z^{(i)} = w^T x^{(i)} + b$ and $\hat{p}^{(i)} = \sigma(z^{(i)})$. Since

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z)),$$

the chain rule simplifies to

$$\frac{\partial \mathcal{L}^{(i)}}{\partial z^{(i)}} = \hat{p}^{(i)} - y^{(i)}.$$

Consequently,

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_i (\hat{p}^{(i)} - y^{(i)}) x_j^{(i)}, \quad \frac{\partial J}{\partial b} = \frac{1}{m} \sum_i (\hat{p}^{(i)} - y^{(i)}).$$

Derivation: The gradient resembles linear regression, but the residual is a probability error created by a sigmoid and cross-entropy pair.

Use: Implement logistic gradient descent with the same vectorization pattern used for linear regression.

6.3 From Probabilities to Decisions

Confusion-matrix metrics

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = 2 \frac{\text{Precision Recall}}{\text{Precision} + \text{Recall}}.$$

Use: Assess thresholded actions when class imbalance or asymmetric error costs make accuracy incomplete.

Notes: Cross-entropy evaluates probability fitting; precision and recall evaluate a specific operating decision. An improved probability model does not force one fixed threshold to remain optimal.

Cue	Reading
Lower threshold τ	Usually raises recall and raises false-positive burden.
Higher threshold τ	Usually raises precision at the risk of missed positives.
Rare positive class	Accuracy can hide failure; examine precision, recall, and base rate.
Calibration need	A score that ranks well may still give unreliable probabilities.

6.4 Cross-Entropy from Maximum Likelihood

Bernoulli likelihood

For $p^{(i)} = \sigma(w^T x^{(i)} + b)$ and $y^{(i)} \in \{0, 1\}$,

$$p(y^{(i)} | x^{(i)}; w, b) = (p^{(i)})^{y^{(i)}} (1 - p^{(i)})^{1-y^{(i)}}.$$

The joint negative average log-likelihood is

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log p^{(i)} + (1 - y^{(i)}) \log(1 - p^{(i)}) \right].$$

Derivation: A confident incorrect probability approaches zero for the observed label, making its log-likelihood very negative and its cross-entropy penalty large. This teaches probability estimates, not merely final labels.

Gradient cancellation

For one observation, $z = w^T x + b$, $p = \sigma(z)$, and $\ell = -y \log p - (1 - y) \log(1 - p)$,

$$\frac{\partial \ell}{\partial p} = -\frac{y}{p} + \frac{1-y}{1-p}, \quad \frac{\partial p}{\partial z} = p(1-p), \quad \frac{\partial \ell}{\partial z} = p - y.$$

Therefore

$$\nabla_w J = \frac{1}{m} \sum_i (p^{(i)} - y^{(i)}) x^{(i)}, \quad \frac{\partial J}{\partial b} = \frac{1}{m} \sum_i (p^{(i)} - y^{(i)}).$$

Derivation: The sigmoid derivative cancels denominators introduced by the log-likelihood. The resulting residual-like error resembles linear regression, but it is the error in probability rather than response value.

6.5 Confusion Metrics and Prevalence

Precision, recall, and F_1

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = 2 \frac{\text{Precision} \text{ Recall}}{\text{Precision} + \text{Recall}}.$$

Why accuracy can be misleading

If only 0.2% of examples are positive, an always-negative model reaches 99.8% accuracy but has zero recall. Under class imbalance, ask which errors matter and how many positive actions are useful, not only how many total labels match.

Cue	Reading
Positive cases rare and important	Inspect precision-recall behavior, not accuracy alone.
False alarms consume expert review	Precision and workload at the deployed threshold matter.
Missed cases create severe harm	Recall or expected cost may dominate.
Need a one-number comparison	Use an agreed operating-cost metric or F_1 only if its balance reflects the task.

6.6 Calibration and Threshold Selection

Calibration meaning

Among examples receiving predicted probability near q , a calibrated model has approximately fraction q positives:

$$\mathbb{P}(Y = 1 \mid \hat{p}(X) \approx q) \approx q.$$

Use: Convert probability predictions into decisions whose costs or resource needs can be estimated.

Notes: A model may rank examples well while being poorly calibrated. Resampling, class weighting, and distribution shift can alter calibration, so thresholds must be validated on a representative distribution.

7 Overfitting and Regularization

Notation and Symbols

Symbol	Meaning in this chapter
J_{train}	Cost evaluated on fitting examples.
J_{val}	Cost evaluated on held-out validation examples.
$\lambda \geq 0$	Regularization strength.
w_j	Feature weight subject to penalty.
$\ w\ _1$	Absolute-weight magnitude $\sum_j w_j $.
$\ w\ _2^2$	Squared Euclidean weight magnitude $\sum_j w_j^2$.
$\rho \in [0, 1]$	Elastic Net mixing proportion assigned to the L_1 term.
d	A complexity choice, such as polynomial degree.

7.1 Underfitting and Overfitting

Complexity and generalization

Cue	Reading
High training error and high validation error	Model or representation is too restricted, optimization is poor, or features lack signal: a high-bias pattern.
Low training error but much higher validation error	Model captures training-specific variation: a high-variance pattern.
Low comparable errors	Model has useful representation for the measured distribution; still check operational metric and data shift.

Use: Diagnose whether added flexibility is likely to help or harm before altering the model.

7.2 L2-Regularized Linear Regression

Penalized objective

$$J_\lambda(w, b) = \frac{1}{2m} \sum_i (f_{w,b}(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2.$$

Notes: The intercept b is conventionally not penalized: shrinking the common baseline normally does not control boundary curvature or feature sensitivity.

Regularized gradient

$$\frac{\partial J_\lambda}{\partial w_j} = \frac{1}{m} \sum_i e^{(i)} x_j^{(i)} + \frac{\lambda}{m} w_j, \quad \frac{\partial J_\lambda}{\partial b} = \frac{1}{m} \sum_i e^{(i)}.$$

The update can be grouped as

$$w_j \leftarrow \left(1 - \alpha \frac{\lambda}{m}\right) w_j - \alpha \frac{1}{m} \sum_i e^{(i)} x_j^{(i)}.$$

Derivation: Each update first contains a multiplicative shrinkage on existing feature weights, then the data-fitting correction. Larger λ trades some training fit for reduced sensitivity to represented features.

7.3 L2-Regularized Logistic Regression

$$J_\lambda(w, b) = -\frac{1}{m} \sum_i \left[y^{(i)} \log \hat{p}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right] + \frac{\lambda}{2m} \sum_j w_j^2,$$
$$\frac{\partial J_\lambda}{\partial w_j} = \frac{1}{m} \sum_i (\hat{p}^{(i)} - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} w_j.$$

Use: Control an overly flexible classification boundary, especially after adding many polynomial or interaction features.

7.4 Choosing Regularization by Evidence

Role of the validation set

Increasing λ almost always raises or maintains training cost for the unpenalized fit term. It is chosen because validation performance may improve:

$$\lambda^* = \arg \min_{\lambda \in \Lambda} J_{\text{val}}(w_\lambda, b_\lambda).$$

Derivation: The fitted parameters depend on λ , so validation compares models produced by competing complexity preferences rather than optimizing weights directly.

Use: Select regularization using held-out behavior, not by seeking the smallest training loss.

Notes: Too large λ drives important weights toward zero and produces underfitting. Regularization reduces variance; it cannot invent missing predictive features or correct a shifted deployment population.

7.5 Regularization as a Complexity Preference

Parameter shrinkage changes the solution

For ridge-regularized linear regression with an augmented intercept excluded from penalization,

$$J_\lambda(w, b) = \frac{1}{2m} \|Xw + b\mathbf{1} - y\|_2^2 + \frac{\lambda}{2m} \|w\|_2^2.$$

Ignoring the separate intercept for compact notation, the stationary solution is

$$w_\lambda = (X^T X + \lambda I)^{-1} X^T y.$$

Derivation: Adding λI increases curvature in weakly supported directions. Coefficients associated with nearly redundant or noisy directions cannot grow cheaply merely to reduce training residuals.

Use: Stabilize models with many correlated or engineered features and reduce sensitivity to finite-sample noise.

7.6 Degree and Regularization Must Be Selected Together

Validation grid

For polynomial degree d and penalty λ , fit each candidate only on training data:

$$(d^*, \lambda^*) = \arg \min_{(d, \lambda) \in \mathcal{G}} J_{\text{val}}(\hat{\theta}_{d, \lambda}).$$

Cue	Reading
High train error and high validation error	Increase useful capacity or improve features; stronger penalty usually worsens bias.
Low train error and much higher validation error	Increase regularization, simplify representation, or add representative data.
Both errors low but deployment failure rises	Diagnose distribution shift, target definition, or operating policy.
Only regularized training objective is reported	Recompute unpenalized predictive error for interpretable bias/variance comparison.

7.7 Practical Regularization Checks

Scale before comparing shrinkage

Because L_2 penalizes coefficient magnitude, feature units influence the amount of effective shrinkage. If one feature is measured in dollars and another in thousands of dollars, equal predictive effects require unequal coefficient magnitudes until scaling is applied. *Use:* Standardize numeric features before treating one global λ as a comparable complexity preference across them.

Regularization does not repair the wrong problem

Cue	Reading
Uninformative available features	Gather or construct signal; shrinking noise does not create information.
Systematic mislabeled outcomes	Fix the data process; regularization merely fits corrupted supervision less aggressively.
Shifted future population	Align evaluation and data collection with deployment; a penalty is not adaptation.
Excessive nonlinear feature variance	Regularization is directly relevant, validated against held-out error.

Notes: A smaller validation loss after regularization is evidence for the tested distribution and metric. It is not a universal claim that the penalized model is more appropriate for every operating condition.

7.8 [Supplemental] Lasso and Elastic Net

L_1 penalty and lasso regression

The lasso replaces squared shrinkage with an absolute-value penalty:

$$\|w\|_1 = \sum_{j=1}^n |w_j|, \quad J_{\lambda}^{\text{lasso}}(w, b) = \frac{1}{2m} \sum_{i=1}^m \left(f_{w, b}(x^{(i)}) - y^{(i)} \right)^2 + \frac{\lambda}{m} \|w\|_1.$$

Notes: The factor multiplying λ is a convention; this sheet uses the same $1/m$ scaling as its regularized regression objectives. The intercept is not penalized.

Subgradient and zero-coordinate sparsity condition

Because $|w_j|$ has a corner at zero, optimization uses

$$\partial|w_j| = \begin{cases} \{1\}, & w_j > 0, \\ [-1, 1], & w_j = 0, \\ \{-1\}, & w_j < 0. \end{cases}$$

Let

$$g_j = \frac{1}{m} \sum_{i=1}^m \left(f_{w,b}(x^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

be the data-loss gradient in coordinate j . At a lasso optimum,

$$0 \in g_j + \frac{\lambda}{m} \partial|w_j|, \quad w_j = 0 \text{ is admissible when } |g_j| \leq \frac{\lambda}{m}.$$

Derivation: An interval of subgradients is available at zero, so a weak enough data-fitting gradient can be cancelled without moving the coefficient away from zero. This is the mathematical source of sparse selected features.

L_1 -regularized logistic objective

$$J_{\lambda}^{L_1}(w, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log \hat{p}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right] + \frac{\lambda}{m} \|w\|_1, \quad \hat{p}^{(i)} = \sigma(w^T x^{(i)} + b).$$

Elastic Net interpolation

$$J_{\lambda, \rho}^{\text{EN}}(w, b) = J_{\text{data}}(w, b) + \frac{\lambda}{m} \left[\rho \|w\|_1 + \frac{1 - \rho}{2} \|w\|_2^2 \right], \quad 0 \leq \rho \leq 1.$$

Cue	Reading
$\rho = 0$	Ridge-style smooth shrinkage; correlated features may remain jointly represented.
$\rho = 1$	Lasso-style sparsity; some coefficients can become exactly zero.
$0 < \rho < 1$	Combine sparsity with squared shrinkage when predictors are numerous or correlated.

Use: Treat the penalty family and its strength as validation-selected design choices; sparsity is useful only if predictive and operational behavior remains acceptable.

8 Theory Checkpoint I

1. A low cost establishes a useful predictor

Statement. A polynomial regression model has nearly zero training mean squared error. Because the objective explicitly punishes large errors, this alone is strong evidence that predictions on new examples will also be accurate.

Answer: False.

Explanation. Mean squared error measures errors on the examples used in its evaluation. A flexible polynomial can drive J_{train} nearly to zero by following sampling noise, especially when degree is high relative to available data. Evidence about new examples requires validation or test error; a large validation gap signals variance even though the optimized training objective is tiny.

2. Scaling changes the best regression relationship

Statement. If gradient descent on linear regression converges using both the raw features and standardized versions of those same features, standardization must produce a different best possible prediction function because the fitted coefficient values are different.

Answer: False.

Explanation. Rescaling reparameterizes the same linear hypothesis class when the transformation is invertible and an intercept is present. The numeric coefficients differ because one unit means something different, while predictions can be transformed to match. Scaling improves optimization conditioning; it is not by itself an increase or decrease in model expressiveness.

3. The logistic threshold is part of the likelihood fit

Statement. After logistic regression has minimized cross-entropy, changing the classification threshold from 0.5 to 0.2 invalidates the learned probability parameters and therefore requires retraining the logistic loss before any decision can be made.

Answer: False.

Explanation. Training estimates probabilities by fitting w, b under cross-entropy. A threshold is a later action rule: $\hat{p} \geq \tau$. Lowering τ moves the operating point, usually increasing recall and false positives, without changing fitted probabilities. Retraining is needed only if the probability model or data is inadequate, not merely because action costs demand a new threshold.

4. Regularization can improve validation performance while worsening fit

Statement. For a high-variance polynomial classifier, selecting a larger nonzero L_2 regularization strength can increase the unregularized training loss while decreasing validation loss.

Answer: True.

Explanation. The penalty deliberately trades fitting freedom for smaller weights and a smoother boundary. Training examples generally prefer more flexibility, so their fit can worsen. New examples can benefit if the original boundary fitted accidental fluctuations. Rejecting a regularized model merely because training loss rose confuses fitting with generalization.

5. Screening capacity and the operating threshold

[Business Scenario]

Statement. A regional clinic uses a logistic model to refer patients for an expensive follow-up test. A newly funded mobile unit doubles follow-up capacity, while disease prevalence and the probability model remain unchanged. Because the fitted probabilities have not changed, lowering the referral threshold cannot be justified mathematically.

Answer: False.

Explanation. The appropriate action threshold depends on the cost of false negatives, the burden of false positives, and available follow-up capacity, not only on probability fit. With more capacity, the clinic can accept more false positives to catch additional true cases. A lower threshold is rational if its incremental detected cases justify the new testing burden; the probability model need not be refitted for this policy change.

6. Demand forecasts under an unseen promotion regime

[Business Scenario]

Statement. A retailer fits regularized regression on ordinary-price weeks and obtains low validation error from a random split of those same weeks. It can treat that validation result as reliable evidence for forecasts during a new nationwide buy-one-get-one campaign, because regularization already protects against overfitting.

Answer: False.

Explanation. Regularization addresses variance within a represented distribution; it does not make unseen promotion mechanics appear in the training data. A random held-out split from ordinary weeks estimates performance on more ordinary weeks, not on an intervention that changes demand behavior. The business should obtain representative promotion data, engineer promotion features with supporting evidence, or explicitly recognize the extrapolation risk.

Part II — Neural Networks, Model Diagnosis, and Tree Ensembles

9 Neural Networks and Forward Propagation

Notation and Symbols

Symbol	Meaning in this chapter
L	Number of layers containing learned parameters.
$n^{[l]}$	Number of units in layer l .
$a^{[0]} = x$	Input feature vector for one example.
$W^{[l]}, b^{[l]}$	Weights and biases mapping layer $l - 1$ to layer l .
$z^{[l]}$	Pre-activation linear score at layer l .
$a^{[l]}$	Activation/output of layer l .
$g^{[l]}$	Activation function used in layer l .

9.1 A Layer as a Reusable Unit

Single dense layer

For a single example, a dense layer computes

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}, \quad a^{[l]} = g^{[l]}(z^{[l]}),$$

with

$$W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}, \quad b^{[l]} \in \mathbb{R}^{n^{[l]}}, \quad a^{[l]} \in \mathbb{R}^{n^{[l]}}.$$

Use: Build a predictor by allowing intermediate units to learn representations before the output unit forms a prediction.

Notes: This chapter includes neural-network material independently because it belongs to the machine-learning syllabus; no separate deep-learning sheet is required.

Neuron view

Unit j in layer l computes

$$z_j^{[l]} = (w_j^{[l]})^T a^{[l-1]} + b_j^{[l]}, \quad a_j^{[l]} = g^{[l]}(z_j^{[l]}).$$

A unit is logistic regression-like when it uses sigmoid, but a network composes many such transformations so hidden representations are learned rather than prescribed.

9.2 Forward Propagation

Layer-by-layer computation

For a network with two hidden layers and a binary output:

$$\begin{aligned} a^{[1]} &= g^{[1]}(W^{[1]}x + b^{[1]}), \\ a^{[2]} &= g^{[2]}(W^{[2]}a^{[1]} + b^{[2]}), \\ \hat{p} &= a^{[3]} = \sigma(W^{[3]}a^{[2]} + b^{[3]}). \end{aligned}$$

Derivation: Each layer's output becomes the next layer's input. The model is not a single linear score because nonlinear $g^{[1]}, g^{[2]}$ interrupt the collapse of matrix products into one linear transformation.

Use: Compute inference once learned parameters are available.

Batch convention

For m examples organized as columns only in this neural-network chapter,

$$A^{[0]} = X_{\text{col}} \in \mathbb{R}^{n^{[0]} \times m}, \quad Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]}, \quad A^{[l]} = g^{[l]}(Z^{[l]}).$$

Notes: The bias vector is broadcast across examples. This local dimension statement prevents confusion with the row-wise data matrix used for regression formulas.

9.3 Choosing an Output Layer

Prediction target	Output activation	Training loss
Continuous value	Linear output	Squared error
Binary event	Sigmoid output	Binary cross-entropy
One of K classes	Softmax vector	Multiclass cross-entropy

Use: Align output semantics and loss; the output layer is selected by the prediction problem, not by aesthetic preference.

9.4 Network Dimensions and Parameter Count

Dense network shape contract

For layer sizes $n_0, n_1, \dots, n_L$, a batch of m column examples obeys

$$\begin{aligned} A^{[0]} &= X \in \mathbb{R}^{n_0 \times m}, \quad W^{[l]} \in \mathbb{R}^{n_l \times n_{l-1}}, \quad b^{[l]} \in \mathbb{R}^{n_l \times 1}, \\ Z^{[l]} &= W^{[l]}A^{[l-1]} + b^{[l]}, \quad A^{[l]} = g^{[l]}(Z^{[l]}) \in \mathbb{R}^{n_l \times m}. \end{aligned}$$

Trainable parameter count

$$\# \text{ params} = \sum_{l=1}^L (n_l n_{l-1} + n_l).$$

Derivation: Each of n_l units receives n_{l-1} incoming weights and one bias. The parameter count grows with layer widths, whereas the batch dimension changes compute and memory but not the number of learned parameters.

Use: Check implementation dimensions and reason about capacity before training.

9.5 From Logistic Unit to Hidden Representation

A neuron as learned feature detector

A single hidden activation is

$$a_j^{[l]} = g^{[l]} \left((w_j^{[l]})^T a^{[l-1]} + b_j^{[l]} \right).$$

The next layer combines these outputs rather than raw measurements. For a demand prediction example, hidden units can represent intermediate patterns such as affordability, awareness, or product fit without requiring those concepts as provided labels. *Notes:* A hidden unit is not guaranteed to match a human-named concept. Its meaning is determined only by how it helps minimize the objective.

Why depth can reuse computations

Suppose a shallow model must represent many combinations of elementary signals. A multilayer network can first compute reusable features $a^{[1]}$, then combine them:

$$x \rightarrow a^{[1]} \rightarrow a^{[2]} \rightarrow \hat{y}.$$

Use: Understand depth as compositional representation, not automatically superior accuracy. Model capacity still must be justified by validation performance and available data.

9.6 Inference Contract

Prediction requires fixed learned quantities

At inference time, weights, biases, preprocessing statistics, and the output decision rule are frozen:

$$x_{\text{raw}} \xrightarrow{\text{training-fitted preprocessing}} a^{[0]} \xrightarrow{\{W^{[l]}, b^{[l]}\}_{l=1}^L} a^{[L]} \xrightarrow{\text{decision rule}} \hat{a}.$$

Cue	Reading
Training cost low but inference preprocessing differs	Predictions are not evaluating the trained mathematical function.
Output is sigmoid probability	A threshold or downstream utility rule still defines the action.
Output is a linear scalar	Use for regression; it is not constrained to a probability range.
New input type or range appears	Treat as distribution shift and validate rather than trusting forward propagation.

9.7 Matrix Vectorization of Multiple Units

Row-wise unit stacking

If unit j has weight row $(w_j)^T$, stacking units into W turns separate dot products into one matrix multiplication:

$$\begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_{n_l} \end{bmatrix} = \begin{bmatrix} (w_1)^T \\ (w_2)^T \\ \vdots \\ (w_{n_l})^T \end{bmatrix} a^{[l-1]} + \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_{n_l} \end{bmatrix} = W^{[l]} a^{[l-1]} + b^{[l]}.$$

Derivation: Vectorization changes neither the network function nor learned objective; it performs the same independent unit computations using optimized matrix operations.

9.8 Convolutional Layers: Local Connectivity

A different layer type

A dense unit receives every activation from the preceding layer. A convolutional unit instead receives activations from a local region and reuses one filter across positions. For a one-channel image and filter $K \in \mathbb{R}^{f \times f}$,

$$Z_{h,w} = \sum_{u=1}^f \sum_{v=1}^f K_{u,v} X_{h+u-1, w+v-1} + b, \quad A_{h,w} = g(Z_{h,w}).$$

Derivation: Local connectivity lets a unit detect a small visual pattern; weight sharing applies that same detector at different locations. A convolutional layer therefore uses fewer parameters than connecting every image pixel to every hidden unit.

Use: Recognize why images motivate a layer structure different from the dense networks used in the earlier formulas.

Notes: This course-level inclusion records the neural-network layer mentioned in the Machine Learning material. Detailed CNN architectures belong to a dedicated deep learning reference rather than this general ML decision sheet.

10 Activations, Multiclass Classification, and Softmax

Notation and Symbols

Symbol	Meaning in this chapter
$g(z)$	Activation function mapping a pre-activation to a unit output.
$\text{ReLU}(z)$	Rectified linear unit $\max(0, z)$.
K	Number of mutually exclusive output classes.
z_k	Logit score for class k .
a_k	Softmax probability assigned to class k .
y_k	One-hot target indicator for class k .

10.1 Hidden-Layer Activations

Common choices

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \text{ReLU}(z) = \max(0, z).$$

Activation	Useful property	Limitation to remember
Sigmoid	Probability-shaped output	Saturates for large $ z $; usually output rather than hidden layer.
tanh	Zero-centered hidden signal	Also saturates for extreme scores.
ReLU	Simple derivative and broad active range	Units with persistently negative scores can become inactive.
Linear	Correct for numeric output	Hidden layers of only linear units collapse to one linear model.

Why nonlinearity is necessary

If every hidden activation is linear,

$$a^{[2]} = W^{[2]}(W^{[1]}x + b^{[1]}) + b^{[2]} = \widetilde{W}x + \widetilde{b}.$$

Derivation: The composition of affine functions is affine. Extra linear hidden layers introduce parameters but not a more expressive decision boundary.

Use: Explain why networks need nonlinear hidden activations to represent complex relationships.

10.2 Multiclass Prediction

Softmax probabilities

For K mutually exclusive classes with logits $z_1, \dots, z_K$,

$$a_k = \frac{e^{z_k}}{\sum_{r=1}^K e^{z_r}}, \quad k = 1, \dots, K, \quad \sum_{k=1}^K a_k = 1.$$

Use: Produce a coherent probability distribution when exactly one class should be selected, such as a product category or handwritten digit.

Notes: For numerical stability implementations subtract $\max_r z_r$ from every logit before exponentiating; probabilities are unchanged.

Multiclass cross-entropy

$$\mathcal{L}(a, y) = - \sum_{k=1}^K y_k \log a_k.$$

When the true class is c , the one-hot vector has $y_c = 1$ and therefore

$$\mathcal{L}(a, y) = -\log a_c.$$

Derivation: Only probability assigned to the true class enters the one-hot loss, but increasing it necessarily takes probability mass from competing classes through the softmax denominator.

Softmax gradient insight

For logit z_k ,

$$\frac{\partial \mathcal{L}}{\partial z_k} = a_k - y_k.$$

Derivation: The compact probability-minus-target error mirrors binary logistic regression and makes backpropagation through a multiclass output tractable.

10.3 Multilabel Is Different

If several labels can be true simultaneously, such as tags on a photograph, use separate sigmoid outputs and binary losses rather than softmax competition:

$$a_k = \sigma(z_k), \quad \mathcal{L} = - \sum_k [y_k \log a_k + (1 - y_k) \log(1 - a_k)].$$

Notes: Softmax asserts mutual exclusivity. Using it for independent labels embeds the wrong problem definition before optimization begins.

10.4 Activation Functions and Their Derivatives

Derivative reference

$g(z)$	$g'(z)$	Typical role
$\sigma(z) = \frac{1}{1 + e^{-z}}$	$\sigma(z)(1 - \sigma(z))$	Binary output probability
$\tanh(z)$	$1 - \tanh^2(z)$	Centered hidden activation
$\text{ReLU}(z) = \max(0, z)$	$\mathbf{1}\{z > 0\}$	Hidden units
z	1	Regression output

Saturation

For sigmoid, $g'(z)$ approaches zero as $|z|$ becomes large. If many hidden units operate in saturated regions, backpropagated gradients can become small:

$$dZ^{[l]} = dA^{[l]} \odot g'^{[l]}(Z^{[l]}).$$

Use: Prefer ReLU-like hidden activations in many dense tasks while matching the output activation to the target semantics.

Notes: ReLU does not remove all optimization risk: units can receive only negative pre-activations and cease to update through zero derivative.

10.5 Stable Softmax and Cross-Entropy

Shift invariance

For logits $z \in \mathbb{R}^K$,

$$p_k = \frac{e^{z_k}}{\sum_{r=1}^K e^{z_r}} = \frac{e^{z_k - c}}{\sum_{r=1}^K e^{z_r - c}} \quad \text{for any constant } c.$$

Select $c = \max_r z_r$ so no exponent exceeds $e^0 = 1$. *Derivation:* Multiplying numerator and denominator by e^{-c} preserves the probabilities but prevents overflow from large positive logits.

Log-sum-exp loss

For true class y ,

$$\ell(z, y) = -\log p_y = -z_y + \log \sum_{r=1}^K e^{z_r}.$$

With $c = \max_r z_r$, compute

$$\ell(z, y) = -(z_y - c) + \log \sum_{r=1}^K e^{z_r - c}.$$

Use: Train multiclass outputs without constructing numerically unstable large exponentials.

Gradient for all logits

Let e_y be the one-hot label vector. Then

$$\nabla_z \ell = p - e_y, \quad \frac{\partial \ell}{\partial z_k} = p_k - \mathbf{1}\{k = y\}.$$

Derivation: Raising the true-class logit reduces loss by increasing its probability; raising any other logit increases competition for total probability. The gradient sums to zero because only relative logits matter.

10.6 Multiclass Versus Multilabel Choice

Cue	Reading
Exactly one category per item	Use one softmax distribution over mutually exclusive labels.
Several independent attributes may apply	Use separate sigmoid outputs and binary cross-entropies.
Classes ordered by severity	Consider whether ordinal costs need a decision rule beyond flat softmax accuracy.
Rare class is operationally critical	Evaluate per-class recall/precision and operating cost, not only average accuracy.

Multilabel objective

For K independent binary labels,

$$\ell_{\text{multi}}(z, y) = -\sum_{k=1}^K [y_k \log \sigma(z_k) + (1 - y_k) \log(1 - \sigma(z_k))].$$

Notes: Softmax would force labels to compete even when several can correctly be present. Choosing the output objective is part of defining the learning problem.

11 Neural Network Training, Backpropagation, and Adam

Notation and Symbols

Symbol	Meaning in this chapter
$J(W, b)$	Average loss of the network over training examples.
$dZ^{[l]}$	Derivative of cost with respect to layer l pre-activation.
$dW^{[l]}, db^{[l]}$	Gradients for learned parameters in layer l .
α	Learning rate.
λ	Weight-decay strength for a regularized network objective.
β_1, β_2	Adam decay factors for first and second moments.
v_t, s_t	Adam moving averages of gradient and squared gradient.
ϵ	Small positive stabilizer in Adam denominator.

11.1 Training Loop

Three training stages

specify architecture and loss $\longrightarrow$ compute forward loss $\longrightarrow$ update parameters from gradients.

A framework automates derivatives, but its mathematical contract remains:

$$W^{[l]} \leftarrow W^{[l]} - \alpha dW^{[l]}, \quad b^{[l]} \leftarrow b^{[l]} - \alpha db^{[l]}.$$

Use: Understand what a high-level training API is performing rather than treating model fitting as an unexplained operation.

11.2 Backpropagation

Output-layer error

For binary sigmoid output with cross-entropy,

$$dZ^{[L]} = A^{[L]} - Y.$$

For a dense layer using column-wise batches,

$$dW^{[l]} = \frac{1}{m} dZ^{[l]} (A^{[l-1]})^T, \quad db^{[l]} = \frac{1}{m} \sum_{i=1}^m dZ^{[l](i)}.$$

Propagating to an earlier layer

$$dA^{[l-1]} = (W^{[l]})^T dZ^{[l]}, \quad dZ^{[l-1]} = dA^{[l-1]} \odot g^{[l-1]'}(Z^{[l-1]}).$$

Derivation: The chain rule sends downstream sensitivity backward through the linear weights and then through each activation derivative. Each $dW^{[l]}$ measures how a small weight change would alter the final cost through all later computations.

Use: Compute all gradients needed by gradient descent without independently perturbing each parameter.

Notes: Shape checks are local safeguards: $dW^{[l]}$ must have the same dimensions as $W^{[l]}$, and $db^{[l]}$ the same dimensions as $b^{[l]}$.

11.3 Optimization with Adam

Moment estimates

Given gradient g_t for any parameter block at iteration t ,

$$\begin{aligned} v_t &= \beta_1 v_{t-1} + (1 - \beta_1) g_t, \\ s_t &= \beta_2 s_{t-1} + (1 - \beta_2) g_t^2, \\ \hat{v}_t &= \frac{v_t}{1 - \beta_1^t}, \quad \hat{s}_t = \frac{s_t}{1 - \beta_2^t}, \\ \theta_t &= \theta_{t-1} - \alpha \frac{\hat{v}_t}{\sqrt{\hat{s}_t} + \epsilon}. \end{aligned}$$

Derivation: The first moment smooths gradient direction; the second moment scales updates down in coordinates with persistently large squared gradients. Bias correction compensates for moment estimates initialized at zero early in training.

Use: Adapt step sizes across parameters when gradients vary substantially in scale.

Notes: Adam can accelerate optimization; it does not replace validation, data quality checks, or a suitable architecture.

11.4 Iterations, Epochs, and Mini-Batches

An epoch processes every training example once; an iteration processes one mini-batch. For mini-batch B_t ,

$$J_{B_t}(\theta) = \frac{1}{|B_t|} \sum_{i \in B_t} \mathcal{L}^{(i)}(\theta).$$

Mini-batches introduce noisy but inexpensive gradients. Their noise can speed progress and scale training, while validation should use a stable evaluation pass.

11.5 Chain-Rule Derivation for a Dense Layer

Local differentials

For one layer,

$$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}, \quad A^{[l]} = g^{[l]}(Z^{[l]}).$$

Given upstream sensitivity $dA^{[l]}$, local propagation is

$$\begin{aligned} dZ^{[l]} &= dA^{[l]} \odot g'^{[l]}(Z^{[l]}), \\ dW^{[l]} &= \frac{1}{m} dZ^{[l]} (A^{[l-1]})^T, \quad db^{[l]} = \frac{1}{m} \sum_{i=1}^m dZ^{[l](i)}, \\ dA^{[l-1]} &= (W^{[l]})^T dZ^{[l]}. \end{aligned}$$

Derivation: Each weight $W_{jk}^{[l]}$ influences unit j 's pre-activation by $A_k^{[l-1]}$; multiplying by the loss sensitivity at that pre-activation and averaging examples yields its gradient. The transpose propagation accumulates all downstream paths that depend on an earlier activation.

Binary cross-entropy output shortcut

For $A^{[L]} = \sigma(Z^{[L]})$ and binary cross-entropy,

$$dZ^{[L]} = A^{[L]} - Y.$$

For softmax with multiclass cross-entropy, the corresponding expression is also

$$dZ^{[L]} = P - Y,$$

where columns of Y are one-hot vectors. *Use:* Recognize the output error that launches backpropagation and verify derived code against dimensions.

11.6 Initialization and Symmetry

Why identical initialization fails

If all neurons in a hidden layer start with identical rows of $W^{[l]}$ and identical biases, they compute identical activations and receive identical gradients. They remain duplicates after every update.

$$W_{1,:}^{[l]} = W_{2,:}^{[l]} \implies dW_{1,:}^{[l]} = dW_{2,:}^{[l]}.$$

Derivation: The objective has no information that distinguishes symmetric units unless their starting parameters break symmetry.

Variance-scaled random weights

For a ReLU layer, a common scale is

$$W_{jk}^{[l]} \sim \mathcal{N}\left(0, \frac{2}{n_{l-1}}\right).$$

Use: Keep activation magnitudes from exploding or vanishing across layers while starting distinct feature detectors.

Notes: Initialization assists optimization; it cannot correct an inappropriate architecture, label problem, or evaluation mismatch.

11.7 Regularization for a Neural Network

Regularized network objective

For layers $l = 1, \dots, L$, a standard L_2 weight-decay objective is

$$J_\lambda(W, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \sum_{l=1}^L \|W^{[l]}\|_F^2.$$

The corresponding weight gradient receives a shrinkage term:

$$dW_\lambda^{[l]} = dW_{\text{data}}^{[l]} + \frac{\lambda}{m} W^{[l]}.$$

Derivation: The penalty makes large weights costly while leaving the bias terms unpenalized in the usual formulation. Increasing λ can reduce variance, but excessive shrinkage raises training error and therefore avoidable bias.

Use: Connect the bias/variance diagnostic choice of changing λ to the actual neural-network objective being optimized.

11.8 Optimization Mechanics

Gradient descent and Adam contrast

Plain mini-batch descent applies one global learning rate:

$$\theta_t = \theta_{t-1} - \alpha g_t.$$

Adam forms first- and second-moment estimates:

$$v_t = \beta_1 v_{t-1} + (1 - \beta_1) g_t, \quad s_t = \beta_2 s_{t-1} + (1 - \beta_2) g_t^2,$$

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{v}_t}{\sqrt{\hat{s}_t} + \varepsilon}.$$

Derivation: Coordinates with persistently large squared gradients receive smaller effective steps; momentum smooths noisy directions. Bias correction matters early because both moving averages begin at zero.

Cue	Reading
Loss fails to decrease at all	Check implementation, labels, activation/loss pairing, and learning rate first.
Loss oscillates severely	Reduce step size or inspect feature/gradient scales.
Training loss falls, validation worsens	Address variance or mismatch; changing optimizer is not the primary diagnosis.
Mini-batch loss is noisy but long-run improves	Noise is expected; track smoothed training and held-out metrics.

11.9 Automatic Differentiation Contract

Framework training does not change mathematics

A library records elementary operations and evaluates exactly the chain-rule gradients of a scalar objective. A correct use still requires:

output semantics $\leftrightarrow$ loss, trainable parameters $\leftrightarrow$ optimizer, validation metric $\leftrightarrow$ decision goal.

Notes: A differentiable training call can optimize the wrong loss perfectly. Always inspect which objective and output interpretation are being differentiated.

12 Evaluation, Bias/Variance, and Learning Curves

Notation and Symbols

Symbol	Meaning in this chapter
J_{train}	Training-set error measured without a regularization penalty for diagnosis.
J_{cv}	Cross-validation or development-set error used for model choices.
J_{test}	Final held-out estimate used after choices are fixed.
J_{baseline}	Reference error achievable by an appropriate comparator.
m_{train}	Number of training examples used in a learning-curve point.
TP, FP, FN, TN	Confusion-matrix counts for thresholded classifications.
$s(x), \tau$	Model score and threshold defining a binary operating point.
TPR, FPR	True-positive and false-positive rates.
AP	Average precision over ordered recall increments.

12.1 Data Splits and Evaluation Roles

Distinct purposes

Split	Purpose
Training set	Learn weights and other fitted parameters.
Cross-validation set	Choose model family, features, degree, regularization, or threshold.
Test set	Estimate final chosen system performance once, without tuning against it.

Use: Prevent optimistic reporting caused by evaluating on examples that shaped the model design.

Notes: A regularization term belongs to training optimization. For bias/variance diagnosis compare prediction losses consistently without adding the penalty to one split but not another.

12.2 Bias and Variance Patterns

Relative errors

Comparing model errors to a useful baseline produces a diagnostic table:

Pattern	Likely diagnosis	Useful next experiments
J_{train} much above baseline	High bias	Richer features/model, reduce regularization, improve optimization.
Small train gap; J_{cv} much higher	High variance	More representative data, regularization, simpler features/model.
Both gaps substantial	Bias and variance	Address training fit, then reduce generalization gap.
Both acceptable but business failure persists	Wrong metric or distribution	Redefine metric/set, perform error analysis.

Derivation: Bias is not simply “large error”: it is avoidable error visible even on training data relative to an attainable baseline. Variance appears as deterioration when moving from fitted data to held-out examples.

12.3 Learning Curves

Error versus training-set size

A learning curve plots

$$J_{\text{train}}(m_{\text{train}}) \quad \text{and} \quad J_{\text{cv}}(m_{\text{train}})$$

while repeating training on increasingly large subsets.

Cue	Reading
Both curves meet at high error	More examples alone usually do not repair high bias; change representation or model.
Wide validation gap that narrows as data grows	Additional representative examples can reduce variance.
Training performance worsens as data grows	Expected: memorizing a larger varied set becomes harder.
Validation distribution differs from deployment	Even well-behaved curves can answer the wrong question.

12.4 Skewed Classes

Precision, recall, and F_1

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2TP}{2TP + FP + FN}.$$

Derivation: When positives are rare, predicting negative almost everywhere can yield excellent accuracy while producing zero useful detections. Precision measures the quality of alerts; recall measures coverage of true cases.

Use: Choose an evaluation metric matching the costs of missed and unnecessary actions.

12.5 Model Selection Pipeline

Three nested decisions

For candidate model family M , hyperparameters λ , and trained parameters θ ,

$$\hat{\theta}_{M,\lambda} = \arg \min_{\theta} J_{\text{train}}(\theta; M, \lambda), \quad (M^*, \lambda^*) = \arg \min_{M,\lambda} J_{\text{cv}}(\hat{\theta}_{M,\lambda}),$$

$J_{\text{test}}(\hat{\theta}_{M^*,\lambda^*})$ reports the final chosen system.

Derivation: Training solves weights within one candidate. Cross-validation selects among candidate systems. Testing measures only after selection; otherwise test feedback becomes additional tuning.

Use: Apply the same discipline when selecting polynomial degree, network size, tree depth, anomaly threshold, or recommender policy.

12.6 Bias and Variance Relative to a Baseline

Avoidable bias and variance gaps

Let E_{base} be a meaningful achievable reference, such as human-level error where it is appropriate. Then

$$\text{avoidable bias} \approx E_{\text{train}} - E_{\text{base}}, \quad \text{variance} \approx E_{\text{cv}} - E_{\text{train}}.$$

Cue	Reading
Large avoidable-bias gap	Increase useful capacity, improve optimization, or improve input representation.
Small bias gap but large variance gap	Gather representative data, regularize, simplify, or use augmentation.
Both gaps large	Address bias first enough to expose whether variance remains limiting.
CV and test differ materially	Suspect evaluation-set mismatch, overuse of CV, or unstable sample sizes.

Notes: The baseline must reflect the task and label quality. Zero error is often not an informative benchmark when observations are ambiguous or labels disagree.

12.7 Learning Curves as an Intervention Test

Training-size sequence

Fit models on increasing training subset sizes $m_1 < \dots < m_q$ using the same design choice, and measure

$$E_{\text{train}}(m_j), \quad E_{\text{cv}}(m_j).$$

Diagnosis

Cue	Reading
Both curves plateau high and close	More examples alone are unlikely to solve high bias; change representation or capacity.
Training error low; CV error high and decreasing	Additional representative data may reduce variance.
Gap persists under more data	Consider regularization, simpler capacity, or mismatch in the evaluation population.
CV improves only after a data source change	The value came from distribution/content, not merely sample count.

12.8 Rare-Event Evaluation

Operating-point metrics

For a threshold τ , define precision and recall as functions of τ :

$$P(\tau) = \frac{TP(\tau)}{TP(\tau) + FP(\tau)}, \quad R(\tau) = \frac{TP(\tau)}{TP(\tau) + FN(\tau)}.$$

Lower thresholds commonly increase recall and reduce precision. A rare-event system therefore needs a selected operating point, not only a global score. *Use*: Select fraud, diagnosis, or quality-control alerting policies against consequences and review capacity.

Notes: Metrics and thresholds are selected on validation data; final claims use a held-out population representative of operation.

12.9 [Supplemental] Threshold Curves and Ranking Metrics

Thresholded score and rates

For a real-valued score $s(x)$, each candidate threshold $\tau \in \mathcal{T}$ defines an operating point:

$$\hat{y}_\tau(x) = \mathbf{1}\{s(x) \geq \tau\}, \quad \text{TPR}(\tau) = \frac{TP(\tau)}{TP(\tau) + FN(\tau)} = R(\tau),$$

$$\text{FPR}(\tau) = \frac{FP(\tau)}{FP(\tau) + TN(\tau)}, \quad \text{TNR}(\tau) = 1 - \text{FPR}(\tau).$$

Notes: Only a calibrated probability score restricts thresholds naturally to $[0, 1]$; a generic ranking score may have any real-valued scale.

ROC curve and ROC-AUC

$$\text{ROC} = \{(\text{FPR}(\tau), \text{TPR}(\tau)) : \tau \in \mathcal{T}\}, \quad \text{AUC}_{\text{ROC}} = \int_0^1 \text{TPR}(u) du, \quad u = \text{FPR}.$$

For independently drawn positive and negative cases,

$$\text{AUC}_{\text{ROC}} = \Pr(s(X^+) > s(X^-)) + \frac{1}{2} \Pr(s(X^+) = s(X^-)).$$

Derivation: Sweeping the threshold measures ranking quality across operating points. The probability form credits half of a tied positive-negative pair and avoids incorrectly claiming that ties are strict successes.

Precision–recall curve and Average precision

$$\text{PR} = \{(R(\tau), P(\tau)) : \tau \in \mathcal{T}\}, \quad \text{AUC}_{\text{PR}} = \int_0^1 P(r) dr.$$

For an ordered set of attainable recall levels $0 = R_0 \leq R_1 \leq \dots \leq R_K$, a standard stepwise summary is

$$\text{AP} = \sum_{k=1}^K (R_k - R_{k-1}) P_k.$$

Cue	Reading
Rare positive class with costly alerts	Inspect PR behavior or AP because false positives directly reduce precision.
Need threshold-free ranking summary	ROC-AUC or AP summarizes ordering, but neither selects an operating threshold.
Need reliable action probabilities	Evaluate calibration separately; a model may rank well while probabilities are inaccurate.

Notes: Software may compute “PR-AUC” using interpolation conventions different from stepwise average precision. Report the convention when comparing systems.

13 ML Strategy and Error Analysis

Notation and Symbols

Symbol	Meaning in this chapter
$\mathcal{D}_{\text{train}}$	Examples used for parameter fitting.
$\mathcal{D}_{\text{dev}}$	Examples and metric used to select improvements.
$\mathcal{D}_{\text{test}}$	Examples reserved for final reporting.
M	Single-number metric used to rank experiments.
C	Satisficing constraint, such as latency below a limit.
$\mathcal{E}$	Manually reviewed set of model errors.

13.1 Evaluation as an Engineering Contract

One ranking metric

A development process moves quickly when candidate systems can be ordered by one declared objective:

$$\theta_{\text{design}}^* = \arg \max_{\theta \in \Theta} M(\theta; \mathcal{D}_{\text{dev}}).$$

Use: Avoid endless disagreement where one system has slightly better precision while another has slightly better recall and no rule says which matters more.

Notes: A metric is a design contract, not a law of nature. Change it when error analysis reveals that it rewards behavior the application does not value.

Optimizing and satisficing metrics

If accuracy is to be optimized while latency must stay tolerable:

$$\max_{\theta} \text{Accuracy}(\theta) \quad \text{subject to} \quad \text{Latency}(\theta) \leq C.$$

Derivation: Collapsing every consideration into one weighted score can hide a hard requirement. A constraint clearly states what is unacceptable regardless of modest accuracy improvement.

13.2 Distribution Alignment

Development and test data

The dev and test sets should represent the population on which decisions will be made. If deployment uses mobile-phone audio but evaluation contains mostly studio recordings, iteration selects the wrong improvements even with impeccable optimization. *Use:* Choose dev/test distributions to match the intended operating environment and refresh them when the product target changes.

Training data may be broader

Large supplementary training data can come from a different source when it improves representations, provided dev and test remain focused on the desired deployment distribution. If performance is poor, a train-dev split can separate variance on the training distribution from mismatch to the target distribution.

13.3 Error Analysis

Manual error categories

Review a manageable sample $\mathcal{E}$ of misclassified development examples and label correctable categories:

$$\text{Fraction}(\text{category } c) = \frac{\#\{e \in \mathcal{E} : e \text{ has category } c\}}{|\mathcal{E}|}.$$

Derivation: If only a tiny fraction of errors arise from a proposed issue, perfectly fixing it caps the possible gain at that tiny fraction. Counting avoids spending weeks solving visible but low-impact failures.

Use: Prioritize data cleanup, feature work, and targeted collection by potential impact rather than anecdotes.

Cue	Reading
Incorrect labels dominate errors	Correct labeling pipeline or relabel important subsets.
A subgroup fails systematically	Collect representative examples and consider subgroup-aware metric review.
Model succeeds on clean cases but fails noisy deployment cases	Target robustness or data mismatch rather than merely increasing depth.
Metric rewards undesirable errors	Revise metric and re-evaluate candidates before tuning further.

13.4 Design the Evaluation Target First

One-number development metric

When many alternatives must be compared quickly, define a metric M before running experiments and choose

$$h^* = \arg \max_{h \in \mathcal{H}} M_{\text{dev}}(h) \quad \text{or} \quad h^* = \arg \min_{h \in \mathcal{H}} M_{\text{dev}}(h).$$

Derivation: Without one agreed ordering, one model can be called better for precision while another is called better for recall, and experimentation no longer supplies a decision.

Use: Accelerate iteration while preserving the ability to revise the metric when it is shown not to represent real value.

Optimizing and satisficing constraints

Some criteria must meet a bound rather than absorb arbitrary trade-offs. A system may be selected as

$$\max_h M_{\text{value}}(h) \quad \text{subject to} \quad M_{\text{latency}}(h) \leq L_{\text{max}}, \quad M_{\text{safety}}(h) \geq S_{\text{min}}.$$

Cue	Reading
Accuracy improves but latency violates a service limit	Candidate fails a satisficing constraint even if its score is larger.
Click metric rewards harmful or misleading material	Change objective/constraints; faster optimization of the proxy is not progress.
No single metric expresses policy restrictions	Rank by an optimizing metric only among systems meeting constraints.
Metric no longer matches current product priorities	Redesign the dev metric and re-evaluate rather than tuning against the old target.

13.5 Development and Test Distribution Design

Target distribution contract

Dev and test examples should be sampled from the population on which future decisions are valued:

$$(X, Y)_{\text{dev}}, (X, Y)_{\text{test}} \sim P_{\text{target}}.$$

Training examples may include broader helpful data, but evaluation should answer the deployed question.

Weighted target mixture

If operation explicitly values settings $r = 1, \dots, R$ in proportions π_r , an evaluation metric may use

$$E_{\text{target}}(h) = \sum_{r=1}^R \pi_r E_r(h), \quad \sum_r \pi_r = 1.$$

Derivation: Weights declare which use cases matter in the final decision. Mixing examples arbitrarily can cause abundant but less-important traffic to dominate model selection.

Notes: The train set need not match target exactly when additional data improves representation, but a mismatch-aware diagnosis must separate its effects.

13.6 Evaluation Resolution Under Iteration

Dev and test size by resolution required

For a classification error estimate $\hat{p}$ based on n approximately independent examples, its standard error is approximately

$$\text{SE}(\hat{p}) \approx \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}.$$

Derivation: Distinguishing two systems whose error rates differ only slightly requires uncertainty substantially smaller than that difference; the required evaluation size therefore depends on the decision resolution, not on a fixed percentage of the training data.

Use: Choose dev/test size to resolve improvements worth acting on; replace the metric or evaluation population only when it orders systems differently from the actual target use, not merely because a preferred experiment lost.

13.7 Avoidable Bias, Variance, and Data Mismatch

Train-dev diagnostic split

If training contains a broader distribution than deployment, create a *train-dev* set drawn from the training distribution but withheld from fitting. Let errors be

$$E_{\text{base}}, \quad E_{\text{train}}, \quad E_{\text{train-dev}}, \quad E_{\text{dev}}.$$

Approximate diagnostic gaps are

$$\begin{aligned} \text{avoidable bias} &\approx E_{\text{train}} - E_{\text{base}}, \\ \text{variance} &\approx E_{\text{train-dev}} - E_{\text{train}}, \\ \text{data mismatch} &\approx E_{\text{dev}} - E_{\text{train-dev}}. \end{aligned}$$

Derivation: Train-dev and dev are both unseen by fitting; their difference isolates a change of distribution more cleanly than comparing dev directly with training error.

Cue	Reading
Large avoidable bias	Improve model/representation/optimization before collecting only more of the same data.
Large variance	Add representative data, regularize, augment, or simplify model capacity.
Large mismatch	Inspect target-specific errors and gather or synthesize target-like training examples.
All measured gaps small but business failure high	Revisit label, metric, constraints, or the task definition itself.

13.8 Reference Baselines and Remedy Selection

Human-level baseline for avoidable bias

Where skilled people can perform the task consistently, define E_{base} from a strong expert or adjudicated team rather than from an untrained observer. It acts as a practical proxy for attainable error:

$$\text{avoidable bias} \approx E_{\text{train}} - E_{\text{base}}.$$

Derivation: A weak human reference exaggerates apparent irreducible error and can make a high-bias system look satisfactory. A strong reference gives a more useful signal for whether additional modeling capacity is worth pursuing.

Cue	Reading
High avoidable bias	Increase useful representation or model capacity, improve optimization, or reduce excessive regularization.
High avoidable bias with visible training errors	Categorize training-set errors too; dev-only analysis does not reveal why the model cannot fit learnable patterns.
High variance on held-out training-distribution examples	Add representative data, apply regularization or early stopping, or simplify only when warranted by evidence or compute constraints.
High data mismatch but moderate variance	Obtain target-like examples or reweight auxiliary data; merely fitting more source-distribution examples may not address deployment errors.

Notes: More data is typically a variance remedy, not the primary response to large avoidable bias. Reducing model size can reduce cost, but it is not an automatic first remedy for variance when regularization and representative data are available. Once the model exceeds a human reference globally, use it mainly to find subsets where experts still reveal fixable errors.

13.9 Manual Error Analysis

Category table

Inspect a sample of incorrectly handled dev examples and label possible causes. For category c , estimate

$$\hat{p}_c = \frac{\#\{\text{examined errors attributed to } c\}}{\#\{\text{examined errors}\}}, \quad \text{maximum possible gain from fixing } c \lesssim \hat{p}_c E_{\text{dev}}.$$

Derivation: Even a perfect fix for a category cannot eliminate errors outside that category. Counting errors prevents investing months in an idea whose ceiling is too small.

Cue	Reading
Many false positives caused by ambiguous labels	Clarify labeling policy and measure inter-annotator agreement.
Errors concentrated in low-light images or noisy audio	Target data collection or augmentation toward that condition.
Errors caused by an impossible missing input	Redesign features or workflow rather than only change the model.
An appealing feature appears in very few errors	Its potential impact may be too small for the next iteration.

Mislabeled evaluation examples

Correct mislabeled dev/test items when their frequency materially affects model selection, and apply a consistent correction process to examples that models get right as well as wrong. *Notes:* Looking only at mislabeled errors of one model biases comparison. A clean, well-specified target set is part of measurement infrastructure.

Eyeball and Blackbox development subsets

When a large development set is repeatedly examined manually, split its roles:

$$\mathcal{D}_{\text{dev}} = \mathcal{D}_{\text{eyeball}} \cup \mathcal{D}_{\text{blackbox}}, \quad \mathcal{D}_{\text{eyeball}} \cap \mathcal{D}_{\text{blackbox}} = \emptyset.$$

$\mathcal{D}_{\text{eyeball}}$ supports manual categorization of failures; $\mathcal{D}_{\text{blackbox}}$ is used for automatic comparison without repeated human inspection. *Derivation:* Manually examining errors adapts design decisions to the examples seen. Keeping an unseen development subset reduces the risk that manual error analysis over-specializes the team's iteration loop.

Use: Preserve rapid error analysis while retaining a less-examined signal for model and hyperparameter comparison.

13.10 Data-Centric Improvement Choices

Data augmentation

An augmentation T is useful when it preserves the desired label and represents plausible future inputs:

$$(x, y) \mapsto (T(x), y), \quad T(x) \sim \text{credible target variation.}$$

Use: Create more evidence for known variation such as lighting, background noise, or benign distortions when suitable for the modality.

Notes: An artificial example easy for humans to accept may still contain repeated artifacts a model can exploit; evaluate on natural target-like data.

Transfer learning

Let f_ϕ be a representation learned on a large related task, and train a new output head g_ψ on target examples:

$$\hat{y} = g_\psi(f_\phi(x)).$$

Initially freeze ϕ when target data are limited; optionally fine-tune selected layers when validation evidence supports it.

Cue	Reading
Small labeled target set; abundant related pretraining	Transfer learning can reduce representation-learning burden.
Large target data and materially different domain	Fine-tuning or training more layers may be justified.
Augmented data repeat unrealistic artifacts	More examples can amplify mismatch; inspect errors on real inputs.
Improvement is on broad training-like data only	Measure on target dev before claiming deployment progress.

13.11 Using Auxiliary and Synthetic Data Under Mismatch

Weighted auxiliary distribution

Suppose T contains target-like examples and A contains broader auxiliary examples. A tunable training objective is

$$J_\beta(\theta) = \frac{1}{|T|} \sum_{i \in T} L_i(\theta) + \beta \frac{1}{|A|} \sum_{i \in A} L_i(\theta), \quad \beta \geq 0.$$

Derivation: Auxiliary data can improve representation while also pulling the model toward the wrong input population or labeling convention. The target dev set selects whether the net effect of β is helpful.

Cue	Reading
Auxiliary data improve target dev performance	Retain them, selecting their effective weight by target evaluation.
Auxiliary labels conflict with the target mapping	Correct, isolate, or downweight them rather than assuming volume is helpful.
Source-distribution score improves while target score deteriorates	Diagnose mismatch; do not call this a deployment improvement.
Target-like examples are scarce but identifiable	Prioritize collection or careful synthesis of those conditions.

Notes: The earlier augmentation rule supplies the corresponding caution for synthetic data: judge it on natural target-like dev examples, not on the generated examples themselves.

13.12 Optimization Verification

Separating scorer failure from search failure

Many systems first assign a score $S_\theta(y \mid x)$ and then search for a candidate output:

$$\hat{y} = \arg \max_{y \in \mathcal{Y}_{\text{search}}} S_\theta(y \mid x).$$

For an erroneous example with a known acceptable output y^* , compare

$$S_\theta(y^* \mid x) \quad \text{with} \quad S_\theta(\hat{y} \mid x).$$

Cue	Reading
$S_\theta(y^*) > S_\theta(\hat{y})$	The scorer preferred the correct result; improve search/optimization.
$S_\theta(y^*) \leq S_\theta(\hat{y})$	Search followed the score; improve learning objective, model, or data.
Desired output is not uniquely defined	Use an acceptable set or human comparison before attributing error.
Policy optimization uses a mis-specified reward	Fix reward/evaluation before debugging the optimizer alone.

Derivation: The test substitutes a known desired output for the search result and asks whether the learned scoring function already ranks it better. It avoids improving the wrong component.

Reward versus policy optimization

For reinforcement learning, let T^* be a known acceptable trajectory and $\hat{T}$ be the trajectory induced by the learned policy under reward R . Compare

$$R(T^*) \quad \text{and} \quad R(\hat{T}).$$

Derivation: A poorly optimized reward and a poorly specified reward demand different fixes. If $R(T^*) > R(\hat{T})$, improve policy optimization or exploration; if an unacceptable $\hat{T}$ scores at least as well, improve reward specification or constraints.

13.13 End-to-End Versus Pipeline Decisions

Where intermediate supervision helps

An end-to-end model directly learns

$$x \mapsto y,$$

while a pipeline introduces intermediate variables

$$x \mapsto z_1 \mapsto z_2 \mapsto y.$$

End-to-end learning can avoid hand-designed bottlenecks when abundant paired (x, y) data exist. A pipeline can exploit intermediate labels, engineering constraints, or separately debuggable components when end-to-end supervision is scarce.

Notes: A rich output such as a sequence or waveform does not itself rule out end-to-end training; availability of informative paired data is the deciding statistical resource.

Error attribution by substitution

If a pipeline predicts $\hat{z}$ before final output, replace it with a correct intermediate value z^* and recompute the downstream output. The improvement estimates how much upstream perfection could help under the existing downstream component.

Use: Allocate work to the component with demonstrated improvement potential rather than the component that is easiest to blame.

Detecting a flawed pipeline

For a pipeline $x \mapsto z_1 \mapsto z_2 \mapsto y$, successively replace $\hat{z}_1, \hat{z}_2$ with oracle values and measure final error:

$$E(\hat{z}_1, \hat{z}_2) \longrightarrow E(z_1^*, \hat{z}_2) \longrightarrow E(z_1^*, z_2^*).$$

Derivation: Large improvement after a substitution localizes the useful intervention. If each component is nearly as strong as human performance given its own input, yet the full system remains far below a human seeing raw input, the pipeline representation has discarded necessary information.

Use: Redesign the intermediate representation or consider a direct model when component polishing cannot recover information removed upstream.

13.14 Ethics and Deployment Reliability

Performance must be evaluated by affected conditions

Aggregate accuracy can conceal subgroup failure. For relevant groups g , monitor

E_g , Precision $_g$, Recall $_g$, action/denial rates under the chosen policy.

Cue	Reading
Training reflects historical harmful decisions	Fitting labels accurately can reproduce harm; review target and constraints.
Rare subgroup has high consequence errors	Collect evidence and define reliability requirements for that condition.
Human reviewers see only model-selected cases	Feedback becomes selection-biased; maintain auditing samples.
Automated action is high stakes	Include fallback, appeal, monitoring, and escalation design, not merely offline score.

Notes: Technical correctness includes the decision process and monitoring context, not merely optimization of an offline aggregate metric.

14 Decision Trees, Entropy, and Information Gain

Notation and Symbols

Symbol	Meaning in this chapter
p_1	Fraction of positive-class examples at a node.
$H(p_1)$	Binary entropy impurity at a node.
$G(p_1)$	Binary Gini impurity at a node.
w_L, w_R	Fractions of node examples sent to left and right children.
IG	Information gain of a proposed split.
s	Candidate feature and threshold split.
d	Tree depth or stopping complexity.

14.1 Recursive Partitioning

Tree prediction

A decision tree recursively applies tests to an input until it reaches a leaf. A classification leaf predicts a majority class or estimated class proportion; a regression leaf predicts an average target value. *Use:* Represent nonlinear rules and feature interactions without constructing polynomial feature combinations manually.

Notes: Trees can be highly interpretable when shallow, but deep trees can vary dramatically with small changes in training data.

14.2 Entropy

Binary impurity

$$H(p_1) = -p_1 \log_2 p_1 - (1 - p_1) \log_2 (1 - p_1),$$

with $0 \log_2 0$ defined as 0.

$$H(0) = H(1) = 0, \quad H(0.5) = 1.$$

Derivation: A pure node has no uncertainty about the class and therefore entropy zero. A balanced node is maximally uncertain and has maximal binary entropy.

Use: Quantify whether a candidate split creates more homogeneous child nodes.

14.3 Information Gain

Reduction in weighted impurity

$$\text{IG}(s) = H(p_1^{\text{root}}) - [w_L H(p_1^L) + w_R H(p_1^R)].$$

Derivation: A split is valuable only if child purity improves after accounting for how many examples enter each child. Creating a tiny pure leaf while leaving almost all examples mixed should not receive the same credit as purifying both substantial groups.

Use: Choose the split with greatest impurity reduction at the current node.

Notes: Recursive tree construction is greedy: it selects the locally best split and does not search all future tree structures for a globally optimal tree.

14.4 Categorical and Continuous Features

Threshold split

For a continuous feature x_j , evaluate candidates of the form

$$x_j \leq t \quad \text{versus} \quad x_j > t,$$

often using midpoints between sorted observed values as threshold candidates. *Use:* Extend information gain from binary indicator features to measured numeric variables.

Regression tree criterion

For numeric targets, a regression split can minimize within-leaf squared error:

$$\text{Reduction}(s) = \text{SSE}_{\text{parent}} - (\text{SSE}_L + \text{SSE}_R), \quad \text{SSE}_A = \sum_{i \in A} (y^{(i)} - \bar{y}_A)^2.$$

Derivation: The best constant prediction inside a leaf under squared error is the leaf mean $\bar{y}_A$; splitting is useful if two means describe the targets better than one.

14.5 Controlling Tree Variance

Cue	Reading
Maximum depth	Limits how many sequential interactions the tree can fit.
Minimum leaf size	Prevents a leaf from representing a very small, noisy subset.
Minimum information gain	Refuses splits whose impurity improvement is negligible.
Pruning or validation choice	Removes complexity not supported by held-out results.

14.6 Leaf Predictions and Split Objectives

Classification leaf probability

For a leaf containing index set S , estimate class probability

$$\hat{p}_S = \frac{1}{|S|} \sum_{i \in S} \mathbf{1}\{y^{(i)} = 1\}, \quad \hat{y}_S = \mathbf{1}\{\hat{p}_S \geq \tau\}.$$

Use: Distinguish the structure-building objective from the eventual operating threshold at a leaf.

Regression leaf value

For squared loss, the optimal constant in a leaf is its average response:

$$c_S^* = \arg \min_c \sum_{i \in S} (y^{(i)} - c)^2 = \frac{1}{|S|} \sum_{i \in S} y^{(i)}.$$

Derivation: Differentiating the leaf squared loss gives $-2 \sum_{i \in S} (y^{(i)} - c) = 0$, so tree regression predicts means within partitions.

14.7 Entropy and Information-Gain Details

Binary entropy behavior

$$H(p) = -p \log_2 p - (1-p) \log_2 (1-p), \quad H(0) = H(1) = 0, \quad H(1/2) = 1.$$

Purity produces no class uncertainty, whereas an evenly mixed node is maximally uncertain in the binary case.

Weighted child entropy

For parent set S split into S_L, S_R ,

$$H_{\text{after}} = \frac{|S_L|}{|S|} H(p_L) + \frac{|S_R|}{|S|} H(p_R), \quad \text{IG} = H(p_S) - H_{\text{after}}.$$

Derivation: A small pure child is not enough: the weighted remainder penalizes a split that leaves most observations mixed. Greedy induction chooses the candidate with highest immediate reduction, not a globally optimal final tree.

14.8 [Supplemental] Alternative Classification Impurity

Gini impurity

For a binary node with positive-class fraction p ,

$$G(p) = 1 - p^2 - (1-p)^2 = 2p(1-p), \quad G(0) = G(1) = 0, \quad G(1/2) = 1/2.$$

A candidate split reduces weighted Gini impurity by

$$\Delta G = G(p_S) - \frac{|S_L|}{|S|} G(p_L) - \frac{|S_R|}{|S|} G(p_R).$$

Classification error impurity

$$E(p) = 1 - \max\{p, 1-p\}.$$

Derivation: Gini and entropy are sensitive to changes in class mixture during tree growth; classification error is flatter and is more useful for describing leaf mistakes than for choosing fine-grained candidate splits.

Notes: The course develops entropy and information gain as the core split rule. These alternative impurity formulas are retained from the supplied tree notes.

14.9 Continuous and Categorical Candidates

Threshold search

For numeric feature x_j , candidate splits use

$$S_L(t) = \{i : x_j^{(i)} \leq t\}, \quad S_R(t) = \{i : x_j^{(i)} > t\},$$

often checking thresholds between adjacent sorted observed values and selecting

$$t^* = \arg \max_t \text{IG}(j, t).$$

Multi-valued categorical features

A categorical feature can split into multiple branches or be grouped into binary partitions. A feature with many rare categories may achieve impressive training purity while producing fragile leaves.

One-hot input encoding for a category

For a categorical input such as ear shape with values {pointy, floppy, oval}, replace one categorical field with indicator features:

$$x_{\text{ear}} = \text{oval} \quad \longmapsto \quad (x_{\text{pointy}}, x_{\text{floppy}}, x_{\text{oval}}) = (0, 0, 1).$$

Derivation: Each indicator becomes a binary candidate split, allowing the same binary tree-building procedure to operate on a feature that originally had more than two possible values.

Use: Represent categorical inputs for tree construction; this is distinct from the one-hot target vector used by softmax classification.

Cue	Reading
Split isolates only a handful of training records	Demand minimum leaf support or prune; purity may be memorization.
Numeric variable permits many thresholds	Validate depth and minimum samples to control selection variance.
Category is unseen at inference	Define routing/fallback behavior before deployment.
Missingness itself predicts outcome	Treat missing indicators/routing as a modeled decision and evaluate shift risk.

14.10 Stopping and Pruning

Pre-pruning rules

Tree growth may stop when

$$\text{depth} \geq d_{\max}, \quad |S| < n_{\min}, \quad \text{IG} < \delta,$$

or a proposed leaf would lack minimum observations.

Complexity-penalized pruning

For tree T with leaves $\mathcal{L}(T)$, a generic pruned objective is

$$J_{\alpha}(T) = \sum_{\ell \in \mathcal{L}(T)} \sum_{i \in S_{\ell}} \mathcal{L}(y^{(i)}, \hat{y}_{\ell}) + \alpha |\mathcal{L}(T)|.$$

Derivation: A split remains only if reduction in fitting loss justifies its added leaf complexity. Validation chooses the relevant complexity level.

Use: Control a single tree's variance and preserve interpretability when a shallow decision rule is required.

Notes: Tree stability matters: a small change in training observations can change early splits and every descendant rule. Ensembles address this sensitivity when individual interpretability is less central.

15 Tree Ensembles and Boosting

Notation and Symbols

Symbol	Meaning in this chapter
B	Number of trees in an ensemble.
$h_b(x)$	Prediction of tree b .
$\mathcal{D}_b$	Bootstrap training sample for tree b .
OOB_i	Trees trained without example i .
$F_m(x)$	Boosted ensemble after stage m .
η	Boosting learning rate or shrinkage factor.
α_t	AdaBoost vote weight for weak classifier t .
$D_t(i)$	AdaBoost sample weight for example i at stage t .
ϵ_t, Z_t	Weighted classification error and sample-weight normalizer.
I_j^{MDI}	Mean decrease in impurity assigned to feature j .

15.1 Bagging and Random Forests

Bootstrap aggregation

Draw B datasets of size m with replacement and train one tree on each:

$$\mathcal{D}_b \sim \text{Bootstrap}(\mathcal{D}), \quad b = 1, \dots, B.$$

For regression and classification respectively,

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B h_b(x), \quad \hat{c}_{\text{bag}}(x) = \arg \max_c \sum_{b=1}^B \mathbf{1}\{h_b(x) = c\}.$$

Derivation: Deep trees are low-bias but high-variance predictors. Averaging varied predictions cancels some instability when tree errors are not perfectly correlated.

Use: Reduce variance of unstable trees without forcing each individual tree to be shallow.

Random feature subsets

A random forest considers only a sampled subset of features at each candidate split. This prevents one dominant predictor from making all trees too alike. *Derivation:* Averaging helps least when model errors are highly correlated. Random feature selection sacrifices some single-tree strength to reduce correlation and can thereby improve ensemble performance.

15.2 Boosted Trees and XGBoost Idea

Sequential correction

Boosting builds an additive function:

$$F_M(x) = F_0(x) + \sum_{m=1}^M \eta h_m(x),$$

where each new small tree focuses on errors or gradients of the current ensemble. *Use:* Build a strong predictor from sequential weak trees, particularly for structured tabular features.

Notes: Unlike bagging, trees are not independently trainable: each stage depends on the residual behavior of earlier stages.

Regularized boosting intuition

A practical boosted-tree implementation can penalize overly complicated added trees and use shrinkage, depth limits, or subsampling. The principle is the same as elsewhere: training correction must be balanced against validation-supported complexity.

15.3 [Supplemental] Out-of-Bag Error

A bootstrap sample leaves out some original examples. Predict example i using only trees that did not train on it:

$$\hat{f}_{\text{OOB}}(x^{(i)}) = \frac{1}{|\text{OOB}_i|} \sum_{b \in \text{OOB}_i} h_b(x^{(i)}).$$

Derivation: For every example, excluded trees behave as held-out predictors with respect to that example. Aggregating these predictions yields an internal generalization estimate without reserving a separate validation subset solely for forest-size exploration.

Notes: Out-of-bag evaluation does not eliminate the need for a final representative test set or prevent over-tuning repeated design decisions against OOB results.

15.4 [Supplemental] Variable Importance

Permutation importance

Measure a baseline validation or OOB score, randomly permute feature j , and measure degradation:

$$I_j = M_{\text{baseline}} - M_{\text{permuted}(j)}$$

when larger M is better. *Use:* Identify features on which fitted predictive performance relies.

Notes: Importance is predictive rather than causal. Correlated features can share or mask importance because one can substitute for another.

15.5 [Supplemental] Gradient Boosting

Squared-error residual fitting

For squared loss with current prediction F_{m-1} , residuals are

$$r_i^{(m)} = y^{(i)} - F_{m-1}(x^{(i)}).$$

Fit a weak tree h_m to residuals and update

$$F_m(x) = F_{m-1}(x) + \eta h_m(x), \quad 0 < \eta \leq 1.$$

Derivation: Under squared error, the negative gradient of loss with respect to the current prediction is the residual. Adding a tree aligned with residuals performs a functional gradient-descent step.

Use: Understand why later trees target what the existing ensemble has not explained.

Notes: Small η often requires more trees but can improve generalization by making each correction conservative.

15.6 [Supplemental] AdaBoost

Reweighted classification errors

Begin with weights $D_1(i) = 1/m$. For weak classifier $h_t(x) \in \{-1, +1\}$,

$$\epsilon_t = \sum_{i=1}^m D_t(i) \mathbf{1}\{h_t(x^{(i)}) \neq y^{(i)}\}, \quad \alpha_t = \frac{1}{2} \log \frac{1 - \epsilon_t}{\epsilon_t}.$$

Update example weights by

$$D_{t+1}(i) = \frac{D_t(i) \exp[-\alpha_t y^{(i)} h_t(x^{(i)})]}{Z_t}, \quad H(x) = \text{sign} \sum_t \alpha_t h_t(x).$$

Derivation: A correctly classified point has $yh = +1$ and is downweighted; an incorrect point has $yh = -1$ and is upweighted. A learner with smaller weighted error receives a larger final vote.

Use: Combine weak classifiers by concentrating later attention on previously misclassified examples.

Notes: Extreme label noise can attract repeated attention and harm the ensemble, because hard examples are not always informative examples.

15.7 [Supplemental] Missing Data and Imbalanced Classes

Missing values

A tree system must define a missing-value policy: imputation fitted on training data, explicit missing indicators, or algorithm-supported default/surrogate routing. Missingness can itself carry information but should not be assumed benign.

Imbalanced outcomes

For rare events, select metrics and training treatment deliberately: class weighting, resampling, threshold tuning, precision-recall evaluation, and calibrated probability checks address different aspects of the problem. *Use:* Avoid confusing an ensemble's raw accuracy with operational value on rare but important events.

Notes: Sampling or class weights can alter the probability interpretation; evaluate calibration and operational thresholds on a distribution representing deployment.

15.8 Why Averaging Can Reduce Variance

Correlated base learners

Suppose regression-tree predictions each have variance σ^2 and pairwise correlation ρ . Averaging B trees gives variance

$$\text{Var} \left(\frac{1}{B} \sum_{b=1}^B T_b(x) \right) = \rho \sigma^2 + \frac{1 - \rho}{B} \sigma^2.$$

Derivation: Increasing B removes only the uncorrelated component. If trees make almost identical errors, averaging cannot substantially improve variance.

Use: Understand why random forests add random feature subsets: they deliberately decorrelate trees while retaining useful strength.

15.9 Bootstrap and Out-of-Bag Reasoning

Probability of exclusion

Sampling m observations with replacement, a specified observation is omitted with probability

$$\left(1 - \frac{1}{m}\right)^m \rightarrow e^{-1} \approx 0.368.$$

Thus each tree has an internal out-of-bag sample comprising roughly 36.8% of training observations.

OOB prediction

Let $\mathcal{B}_{-i}$ denote trees whose bootstrap sample omitted observation i . For regression,

$$\hat{y}_{\text{OOB}}^{(i)} = \frac{1}{|\mathcal{B}_{-i}|} \sum_{b \in \mathcal{B}_{-i}} T_b(x^{(i)}),$$

with majority vote used for classification. *Use:* Obtain a training-internal generalization estimate for bagged trees and tune some forest design choices without reserving each tree's own extra split.

Notes: OOB is repeatedly used while building the forest and remains tied to the training distribution. Retain a final target-distribution evaluation for selected systems.

15.10 Variable Importance Is Predictive, Not Causal

Permutation importance

For feature j , measure held-out performance M , permute only feature j , and compute

$$I_j = M_{\text{original}} - M_{\text{permuted}(j)}$$

when higher M is better. *Derivation:* Permutation breaks the feature's association with the outcome while preserving the fitted model. A performance drop indicates predictive dependence of this model on that variable.

Cue	Reading
Two features contain similar signal	Importance can be divided or masked because either substitutes for the other.
Feature is a proxy for protected or post-outcome information	Predictive importance does not justify use or causal interpretation.
Permutation creates impossible combinations	Measured degradation may reflect unrealistic inputs; interpret cautiously.
Importance differs across operational groups	Inspect reliability and policy impact by relevant conditions.

Mean decrease in impurity

For split node q in tree b , let N_q be its sample count, let ΔI_q be its weighted reduction in Gini impurity or entropy, and let $j(q)$ be the selected feature. A forest impurity importance can be written

$$I_j^{\text{MDI}} = \frac{1}{B} \sum_{b=1}^B \sum_{\substack{q \in T_b \\ j(q)=j}} \frac{N_q}{N} \Delta I_q.$$

Derivation: Each split credits the selected variable for the impurity it removes, weighted by how many training examples reach that node, then averages credit across trees.

Notes: MDI is computed from fitted-tree splits and can favor features with many candidate cut points or categories. Permutation importance answers a different, held-out predictive-dependence question.

15.11 Gradient Boosting as Functional Gradient Descent

Boosting initialization

Before fitting residual corrections, choose a constant baseline

$$F_0(x) = \arg \min_c \sum_{i=1}^m \mathcal{L}(y^{(i)}, c).$$

For squared-error regression, $F_0(x) = \bar{y}$.

General differentiable loss

For additive model $F_m(x) = F_{m-1}(x) + \eta h_m(x)$, define pseudo-residuals

$$r_i^{(m)} = - \left. \frac{\partial \mathcal{L}(y^{(i)}, F(x^{(i)}))}{\partial F(x^{(i)})} \right|_{F=F_{m-1}}.$$

Fit a weak tree h_m to approximate these residual directions and update

$$F_m(x) = F_{m-1}(x) + \eta h_m(x).$$

Derivation: For squared error, $\mathcal{L} = \frac{1}{2}(y - F)^2$ and $r_i^{(m)} = y^{(i)} - F_{m-1}(x^{(i)})$, recovering residual fitting. For other losses, the same reasoning targets negative gradients rather than raw residuals.

Complexity controls

small tree depth, learning rate η , number of rounds M , subsampling/regularization

act together. A smaller η generally needs more boosting rounds; validation or early stopping selects the useful combination.

Use: Explain boosted-tree strength on structured tabular data and why sequential overfitting must be monitored.

15.12 AdaBoost Through Exponential Loss

Additive classifier

For labels $y_i \in \{-1, +1\}$, let

$$F_T(x) = \sum_{t=1}^T \alpha_t h_t(x), \quad \mathcal{J}(F) = \sum_i e^{-y_i F(x_i)}.$$

At round t , the weight

$$D_t(i) \propto e^{-y_i F_{t-1}(x_i)}$$

is larger for points currently assigned low or wrong margin. Minimizing the next stage over a binary weak learner gives

$$\alpha_t = \frac{1}{2} \log \frac{1 - \epsilon_t}{\epsilon_t}.$$

Weighted error and final classifier

$$\epsilon_t = \frac{\sum_{i=1}^m D_t(i) \mathbf{1}\{y_i \neq h_t(x_i)\}}{\sum_{i=1}^m D_t(i)}, \quad D_{t+1}(i) = \frac{D_t(i) e^{-\alpha_t y_i h_t(x_i)}}{Z_t},$$
$$\hat{y}(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right).$$

Derivation: When $\epsilon_t < 1/2$, the weak learner receives a positive vote; as weighted error approaches zero its influence grows. Misclassified cases receive larger subsequent exponential weight because their margin is negative.

Notes: The same mechanism that focuses on difficult examples can over-focus on incorrect labels or irreducible outliers.

15.13 Missing Data and Imbalanced Outcomes in Ensembles

Missing-value policy

For each feature used in splitting, choose a training-fitted policy: impute, create an explicit missingness indicator, route missing observations by a learned default branch, or use an algorithm with defined native handling.

Rare-class objective

Class weighting replaces an unweighted classification objective by

$$J_w = \frac{1}{m} \sum_{i=1}^m c_{y^{(i)}} \mathcal{L}(\hat{p}^{(i)}, y^{(i)}), \quad c_1 > c_0$$

when positive mistakes deserve more fitting attention.

Cue	Reading
Resampling or class weights improve recall	Recheck calibration and choose an operating threshold on representative validation data.
Missingness rate changes in production	Treat it as drift; even native routing learned under old missingness can fail.
Rare fraud labels are delayed or incomplete	Offline precision/recall may be biased by label availability; design audits.
Forest or booster accuracy is high on imbalance	Demand rare-class metrics and consequence-sensitive evaluation.

15.14 Tree Ensembles Versus Neural Networks

Algorithm-choice guide

Cue	Reading
Structured tabular data	Tree ensembles, particularly boosted trees, are strong baselines and often train quickly.
Images, audio, or text	Neural networks are generally preferred because they learn representations for unstructured inputs.
Mixed structured and unstructured inputs	Neural architectures can combine learned representations with tabular variables.
Very constrained inference or explicit small rules	A small decision tree may be chosen for speed or inspectability.

Notes: This is a starting preference, not a theorem: use validation under the intended metric and deployment constraints. Neural networks can benefit from transfer learning; ensembles remain highly effective for many spreadsheet-like prediction problems.

16 Theory Checkpoint II

1. Extra linear layers create nonlinear representations

Statement. A neural network with three hidden layers but linear activations in every hidden unit can learn nonlinear feature interactions because the successive matrices are learned independently during backpropagation.

Answer: False.

Explanation. Regardless of how the matrices were learned, the composition of affine mappings is another affine mapping:

$$W^{[3]}(W^{[2]}(W^{[1]}x + b^{[1]}) + b^{[2]}) + b^{[3]} = \widetilde{W}x + \widetilde{b}.$$

Independent parameters do not create nonlinear expressiveness unless nonlinear activation functions intervene.

2. Learning curves can distinguish a data remedy from a complexity remedy

Statement. If training and validation errors have already converged to approximately the same unacceptably high value as training-set size grows, collecting substantially more examples from the same distribution is ordinarily the first remedy for high variance.

Answer: False.

Explanation. The small generalization gap is not the signature of high variance; the high shared error points to bias or insufficient representation. Additional similar data is most promising when a sizeable validation gap is narrowing with sample size. Here a richer model or better features, reduced excessive regularization, or optimization review is the more direct experiment.

3. Greedy information gain certifies the globally best tree

Statement. At each decision-tree node, selecting the split with maximum information gain guarantees that the resulting full tree has the minimum possible validation error among all trees of the same maximum depth.

Answer: False.

Explanation. Information gain is a local training-data impurity reduction. A split that looks best immediately can prevent a superior later partition, and it does not optimize held-out error. Depth, minimum leaf size, pruning, and validation selection remain necessary because greedy recursive partitioning is not a global validation optimizer.

4. OOB evidence and final test evidence are interchangeable

Statement. For a random forest, out-of-bag error can provide a useful estimate while building the model, but repeatedly choosing features and hyperparameters to maximize OOB performance can make a separate final test set valuable again.

Answer: True.

Explanation. An OOB prediction excludes each example from the individual trees used to predict it, so it is a principled internal evaluation for one fitting design. Once many design choices are selected to improve that same OOB score, information from the training collection has shaped the final model-selection procedure. A separately representative untouched test set measures the chosen procedure more honestly.

5. Fraud alerts and accuracy

[Business Scenario]

Statement. A payment network faces fraud in 0.08 percent of transactions. A new random-forest classifier increases held-out accuracy from 99.90 percent to 99.93 percent, so the fraud team can conclude it is operationally superior without examining thresholds, precision, recall, or investigation capacity.

Answer: False.

Explanation. With extreme imbalance, tiny movements among legitimate transactions dominate accuracy. The new classifier could produce fewer fraud catches while improving accuracy by suppressing alerts. The team needs an operating threshold and metrics linked to prevented loss and review capacity, such as precision-recall behavior, expected monetary value, and calibration. A forest is not valuable merely because majority-class predictions improve.

6. Visual inspection and a shifted error distribution

[Business Scenario]

Statement. A manufacturer trains a defect detector on evenly illuminated factory photographs and selects a neural network using a matched validation set. At launch, most missed defects occur on a new night shift with dim cameras. Since the validation metric was properly held out during training, the correct first action is simply to increase network depth rather than revisit evaluation data or error categories.

Answer: False.

Explanation. The original validation set estimated performance under the old imaging environment, not the newly important deployment distribution. Manual error analysis has already identified a systematic category tied to camera conditions. The first useful steps are to align development evaluation with real deployment, acquire or correct dim-light examples, and then compare remedies. Increasing depth before fixing the evaluation target can optimize a model for the wrong problem.

Part III — Clustering, Recommendations, and Reinforcement Learning

17 K-means Clustering

Notation and Symbols

Symbol	Meaning in this chapter
$x^{(i)} \in \mathbb{R}^n$	Unlabeled example i .
K	Chosen number of clusters.
$\mu_k \in \mathbb{R}^n$	Centroid representing cluster k .
$c^{(i)}$	Index of centroid assigned to example i .
$J(c, \mu)$	Within-cluster distortion objective.

17.1 Unsupervised Grouping Objective

Distortion cost

K-means seeks assignments and centroids that minimize average squared distance:

$$J(c, \mu) = \frac{1}{m} \sum_{i=1}^m \|x^{(i)} - \mu_{c^{(i)}}\|_2^2.$$

Use: Discover compact groups when labels are unavailable and Euclidean proximity in the represented features corresponds to meaningful similarity.

Notes: Clusters are produced by an objective, not discovered as objective truth. Feature choice and scaling define what “near” means.

17.2 Alternating Optimization

Assignment step

Holding centroids fixed, choose the closest centroid for each example:

$$c^{(i)} \leftarrow \arg \min_{k \in \{1, \dots, K\}} \|x^{(i)} - \mu_k\|_2^2.$$

Centroid update step

Holding assignments fixed, replace each centroid by its assigned mean:

$$\mu_k \leftarrow \frac{\sum_{i=1}^m \mathbf{1}\{c^{(i)} = k\} x^{(i)}}{\sum_{i=1}^m \mathbf{1}\{c^{(i)} = k\}}.$$

Derivation: For fixed assignments, the vector minimizing the sum of squared distances in one cluster is its component-wise mean. For fixed centroids, selecting the nearest one minimizes each example’s contribution. Each step therefore cannot increase distortion, though the final local minimum may depend on initialization.

17.3 Initialization and Model Choice

Random initialization

Select K distinct training examples as initial centroids and run K-means multiple times; retain the run with smallest distortion:

$$(c^*, \mu^*) = \arg \min_{r \in \{1, \dots, R\}} J(c_r, \mu_r).$$

Use: Reduce the risk of an unfortunate local minimum.

Choosing K

The distortion J never increases when K grows, because extra centroids can be ignored or used. An elbow plot can suggest diminishing returns, but an operational goal often decides more clearly: customer tiers, image-compression budget, or a desired number of groups for action. *Notes:* Selecting K only because it produces the lowest training distortion would select the largest allowable K , which does not answer whether clusters are useful.

17.4 Feature Scaling and Interpretation

Cue	Reading
Unscaled feature with large numeric range	It dominates squared distance even if not semantically dominant.
Elongated or nonconvex natural groups	K-means spherical-distance partitions may be misleading.
Very different cluster sizes/densities	Centroid partition may split a broad group or absorb a small group.
Actionable segments required	Evaluate downstream utility, not merely a low distortion value.

17.5 Objective and Monotonic Alternation

Cluster distortion

For assignment $c^{(i)} \in \{1, \dots, K\}$ and centroids $\mu_1, \dots, \mu_K$,

$$J(c, \mu) = \frac{1}{m} \sum_{i=1}^m \|x^{(i)} - \mu_{c^{(i)}}\|_2^2.$$

Assignment minimizes with fixed centroids

$$c^{(i)} \leftarrow \arg \min_{k \in \{1, \dots, K\}} \|x^{(i)} - \mu_k\|_2^2.$$

Each point selects the nearest current centroid, so no single assignment change can raise J while μ is held fixed.

Mean minimizes with fixed assignments

For $S_k = \{i : c^{(i)} = k\}$,

$$\mu_k \leftarrow \frac{1}{|S_k|} \sum_{i \in S_k} x^{(i)}.$$

Derivation: Differentiating $\sum_{i \in S_k} \|x^{(i)} - \mu_k\|^2$ with respect to μ_k sets the centroid to the cluster mean. Therefore each alternating step does not increase distortion, though the final local minimum depends on initialization.

17.6 Initialization and Local Minima

Multiple restarts

Run K-means with independently chosen initial centroids $\mu^{(0,r)}$ for $r = 1, \dots, R$ and retain

$$r^* = \arg \min_r J(c^{(r)}, \mu^{(r)}).$$

Use: Reduce sensitivity to an unlucky initial partition when K is moderate and repeated fitting is computationally feasible.

Notes: Choosing the best restart for a fixed K is coherent because all restarts optimize the same distortion objective. It does not prove that the chosen K is semantically meaningful.

Empty clusters

If $S_k = \emptyset$, its mean is undefined. A practical algorithm must reinitialize that centroid, for example to a currently poorly represented point, and continue fitting.

17.7 Scaling and Geometry

Distance determines meaning

After standardizing numeric features,

$$z_j = \frac{x_j - \mu_j}{s_j}, \quad \|z^{(i)} - \nu_k\|_2^2 = \sum_j \left(\frac{x_j^{(i)} - \tilde{\mu}_{k,j}}{s_j} \right)^2.$$

Derivation: Without scaling, features measured on large numeric ranges dominate Euclidean distance and therefore dominate cluster assignments.

Cue	Reading
Features differ only by arbitrary measurement units	Standardize before treating Euclidean distance as similarity.
Some feature should genuinely dominate similarity	Encode that decision through a documented weight, not accidental units.
Clusters are elongated or non-spherical	K-means compact spherical geometry may be unsuitable.
Business segmentation requires actionability	Select K using interpretability and downstream value, not distortion alone.

17.8 Choosing the Number of Clusters

Elbow reasoning

Define optimized distortion at K :

$$J^*(K) = \min_{c, \mu} J(c, \mu).$$

Because adding centroids cannot worsen the best possible assignment,

$$J^*(K+1) \leq J^*(K).$$

Derivation: Minimum distortion always rewards more clusters; selecting K solely by its smallest value chooses the largest permitted K . An elbow is only a heuristic for diminishing returns and may be absent.

Use: Combine curve shape with the intended use of the groups, stability across samples, and whether distinct decisions can actually be made for the segments.

18 Anomaly Detection

Notation and Symbols

Symbol	Meaning in this chapter
$x \in \mathbb{R}^n$	Feature vector describing a candidate observation.
μ_j, σ_j^2	Estimated mean and variance of feature j on normal data.
$p(x)$	Estimated probability density under ordinary behavior.
ϵ	Density threshold below which an observation is flagged.
$y \in \{0, 1\}$	Evaluation label, with 1 denoting an anomaly when available.

18.1 Density-Based Detection

Independent Gaussian feature model

For normal examples, estimate

$$\mu_j = \frac{1}{m} \sum_{i=1}^m x_j^{(i)}, \quad \sigma_j^2 = \frac{1}{m} \sum_{i=1}^m (x_j^{(i)} - \mu_j)^2.$$

Assuming represented features are modeled independently,

$$p(x) = \prod_{j=1}^n \frac{1}{\sqrt{2\pi\sigma_j^2}} \exp \left[-\frac{(x_j - \mu_j)^2}{2\sigma_j^2} \right].$$

Flag rule

$$\hat{y} = \begin{cases} 1, & p(x) < \epsilon, \\ 0, & p(x) \geq \epsilon. \end{cases}$$

Derivation: The model is fitted primarily to ordinary behavior. A point is suspicious when normal behavior assigns it very small density, whether or not an identical anomaly type has appeared before.

Use: Detect rare unusual patterns when anomaly varieties are diverse or positive examples are scarce.

18.2 Selecting Features and Thresholds

Feature design

Good anomaly features cause abnormal examples to occupy low-density regions under the normal model. If a feature is extremely skewed, transformations such as $\log(x + c)$ may yield a distribution better represented by a Gaussian.

Development-set selection

When a small labelled anomaly development set exists, choose ϵ using an imbalanced-class metric:

$$\epsilon^* = \arg \max_{\epsilon \in E} F_1(\hat{y}_\epsilon, y)$$

or an application-specific value measure. *Use:* Turn a density score into an operating decision using evidence about important false positives and false negatives.

Notes: The normal-density parameters should be fitted from ordinary training examples, while the threshold can use held-out labelled anomalies.

18.3 Anomaly Detection versus Supervised Classification

Prefer anomaly detection when	Prefer supervised classification when
Positive examples are very rare and future anomaly types may differ.	There are enough representative positive examples and future positives resemble past ones.
The central task is recognizing normal behavior and flagging departures.	The central task is learning a repeatable discriminative boundary between known classes.

18.4 Estimating a Gaussian Normality Model

Maximum-likelihood parameters

Using normal training examples for feature j ,

$$\mu_j = \frac{1}{m} \sum_{i=1}^m x_j^{(i)}, \quad \sigma_j^2 = \frac{1}{m} \sum_{i=1}^m (x_j^{(i)} - \mu_j)^2.$$

With conditional independence approximation,

$$p(x) = \prod_{j=1}^n \frac{1}{\sqrt{2\pi\sigma_j^2}} \exp \left[-\frac{(x_j - \mu_j)^2}{2\sigma_j^2} \right].$$

Derivation: Taking logs converts the product into a sum:

$$\log p(x) = -\frac{1}{2} \sum_j \left[\log(2\pi\sigma_j^2) + \frac{(x_j - \mu_j)^2}{\sigma_j^2} \right].$$

Low density occurs when standardized deviations are collectively unlikely under the normal-behavior model.

Use: Score unusual observations when examples of future failure modes are sparse or diverse.

18.5 Threshold Selection With Labelled Evaluation Cases

Alarm rule

$$\hat{y} = \begin{cases} 1, & p(x) < \varepsilon \quad (\text{anomaly}), \\ 0, & p(x) \geq \varepsilon \quad (\text{normal}). \end{cases}$$

When a small labelled validation set contains both ordinary and anomalous cases, select

$$\varepsilon^* = \arg \max_{\varepsilon \in \mathcal{E}} F_1(\varepsilon) \quad \text{or minimize a specified cost.}$$

Derivation: Training fits density of normal behavior without needing many failure classes. The labelled validation cases determine how rare a density must be before the alarm is operationally worthwhile.

Cue	Reading
Numerous repeatable positive failure examples	Supervised classification can learn failure-specific boundaries.
Future anomalies may be unlike known anomalies	Modeling normality may generalize better to novel deviations.
Failure has high cost and review is cheap	Choose a threshold favoring recall, validated against workload.
Validation includes no representative anomalies	Threshold quality is uncertain; obtain labelled cases or simulate cautiously.

18.6 Feature Engineering for Anomaly Scores

Transform skewed ordinary behavior

If a positive-valued feature is heavily skewed, transforms such as

$$x \mapsto \log(x + \delta), \quad x \mapsto x^{1/2}$$

can make normal observations closer to Gaussian and make low-density tails more meaningful.

Create features exposing unusual relations

Anomaly may not appear in either raw variable alone but in a relation, such as

$$x_{\text{ratio}} = \frac{\text{CPU load}}{\text{network traffic} + \delta}.$$

Use: Construct features where abnormal operations occupy a low-density region, then re-evaluate threshold performance on held-out labelled cases.

19 Collaborative Filtering

Notation and Symbols

Symbol	Meaning in this chapter
n_u	Number of users.
n_m	Number of items or movies.
n	Dimension of a learned preference/item feature vector.
$x^{(i)}$	Feature vector learned for item i .
$w^{(j)}, b^{(j)}$	Preference vector and bias for user j .
$y^{(i,j)}$	Rating user j gave item i .
$r(i, j)$	Indicator that rating $y^{(i,j)}$ was observed.
λ	Regularization strength for learned latent factors.
$d(i, k)$	Squared latent-space distance between items i and k .

19.1 Predicting Ratings from Interactions

Rating model

For item i and user j ,

$$\hat{y}^{(i,j)} = (w^{(j)})^T x^{(i)} + b^{(j)}.$$

Use: Recommend items by learning both what an item represents and what a user values from a partially observed ratings matrix.

Notes: The learned feature dimensions need not carry human-provided labels; they are latent coordinates useful for reconstructing observed preferences.

19.2 Collaborative Filtering Objective

Observed-entry squared loss

$$J(x, w, b) = \frac{1}{2} \sum_{i=1}^{n_m} \sum_{j=1}^{n_u} r(i, j) \left(\hat{y}^{(i,j)} - y^{(i,j)} \right)^2 + \frac{\lambda}{2} \sum_i \|x^{(i)}\|_2^2 + \frac{\lambda}{2} \sum_j \|w^{(j)}\|_2^2.$$

Derivation: Only observed ratings create fit residuals; otherwise the algorithm would incorrectly treat an unrated item as a disliked item. Regularization resolves some ambiguity and discourages extreme latent vectors that fit sparse ratings.

Use: Jointly learn item embeddings and user preference vectors from interaction patterns.

Parameter updates

For a particular item vector and user vector,

$$\begin{aligned} \frac{\partial J}{\partial x^{(i)}} &= \sum_{j:r(i,j)=1} (\hat{y}^{(i,j)} - y^{(i,j)}) w^{(j)} + \lambda x^{(i)}, \\ \frac{\partial J}{\partial w^{(j)}} &= \sum_{i:r(i,j)=1} (\hat{y}^{(i,j)} - y^{(i,j)}) x^{(i)} + \lambda w^{(j)}. \end{aligned}$$

Derivation: An item representation moves toward users whose observed ratings it underpredicts and away when it overpredicts, weighted by those users' preference directions; user vectors move symmetrically.

19.3 Mean Normalization and Cold Start

User/item baseline

If item means μ_i are removed before fitting,

$$y_{\text{norm}}^{(i,j)} = y^{(i,j)} - \mu_i, \quad \hat{y}^{(i,j)} = (w^{(j)})^T x^{(i)} + b^{(j)} + \mu_i.$$

Use: Give a new user with few ratings a defensible baseline rather than arbitrary untrained personalized scores.

Notes: Pure collaborative filtering struggles for entirely new items or users because interactions are absent; content features or onboarding information address this cold-start problem.

19.4 Binary Feedback

When interactions are clicks or purchases rather than ratings, use a probability model:

$$\hat{p}^{(i,j)} = \sigma((w^{(j)})^T x^{(i)} + b^{(j)}),$$

with binary cross-entropy summed only over appropriately sampled or defined exposure events. An unseen item is not automatically a negative preference if the user was never shown it.

19.5 Jointly Learning Item and User Factors

Low-dimensional interaction model

For item vector $x^{(i)} \in \mathbb{R}^k$, user vector $\theta^{(j)} \in \mathbb{R}^k$, and optional user bias $b^{(j)}$,

$$\hat{y}^{(i,j)} = (\theta^{(j)})^T x^{(i)} + b^{(j)}.$$

Let $r(i, j) = 1$ only when rating $y^{(i,j)}$ is observed:

$$J(X, \Theta, b) = \frac{1}{2} \sum_{(i,j):r(i,j)=1} \left(\hat{y}^{(i,j)} - y^{(i,j)} \right)^2 + \frac{\lambda}{2} \left(\sum_i \|x^{(i)}\|^2 + \sum_j \|\theta^{(j)}\|^2 \right).$$

Derivation: Unlike content features, both item and user representations can be unknown. Shared observed interactions couple their learning: users who rate similar items similarly acquire compatible vectors.

Gradients for one interaction structure

Let $e^{(i,j)} = \hat{y}^{(i,j)} - y^{(i,j)}$. Then

$$\frac{\partial J}{\partial x^{(i)}} = \sum_{j:r(i,j)=1} e^{(i,j)} \theta^{(j)} + \lambda x^{(i)},$$

$$\frac{\partial J}{\partial \theta^{(j)}} = \sum_{i:r(i,j)=1} e^{(i,j)} x^{(i)} + \lambda \theta^{(j)}, \quad \frac{\partial J}{\partial b^{(j)}} = \sum_{i:r(i,j)=1} e^{(i,j)}.$$

Use: See why a rating updates both the representation of an item and the inferred taste of its user.

19.6 Retrieving Similar Items

Related items in latent space

Collaborative filtering learns an item vector even when its coordinates were not human-named. For reference item i , retrieve items k with small squared distance:

$$d(i, k) = \|x^{(k)} - x^{(i)}\|_2^2 = \sum_{\ell=1}^n \left(x_{\ell}^{(k)} - x_{\ell}^{(i)}\right)^2.$$

Use: Produce a related-items list from the same latent features learned for rating prediction. This is separate from scoring items for one particular user.

19.7 Missing Is Not Negative

Observed-entry masking

The indicator $r(i, j)$ prevents unobserved explicit ratings from contributing to squared loss:

$$J = \frac{1}{2} \sum_{i,j} r(i, j) \left(\hat{y}^{(i,j)} - y^{(i,j)}\right)^2 + \dots$$

Derivation: A blank explicit rating records absence of observation, not necessarily dislike. Penalizing every blank as zero teaches popularity/exposure artifacts as negative preference.

Cue	Reading
Explicit stars are missing	Exclude unknown entries from rating loss.
Clicks or purchases are observed positives	Define negative sampling/exposure assumptions explicitly.
Catalog item has no interactions	Collaborative factors cannot estimate it well; use content features or exploration.
New user has no interactions	Request initial signals, use context/popularity carefully, or explore.

19.8 Mean Normalization

Item baseline correction

For item i , compute its mean over observed ratings

$$\mu_i = \frac{\sum_{j:r(i,j)=1} y^{(i,j)}}{\sum_j r(i, j)}, \quad \tilde{y}^{(i,j)} = y^{(i,j)} - \mu_i.$$

Train on $\tilde{y}$ and predict

$$\hat{y}^{(i,j)} = (\theta^{(j)})^T x^{(i)} + b^{(j)} + \mu_i.$$

Use: Represent general item popularity separately from user-specific affinity, especially for a user with few ratings.

19.9 Binary Feedback Objective

Engagement probability

For implicit binary target $y^{(i,j)} \in \{0, 1\}$,

$$p^{(i,j)} = \sigma \left((\theta^{(j)})^T x^{(i)} + b^{(j)} \right),$$

$$J = - \sum_{(i,j) \in \mathcal{D}} \left[y^{(i,j)} \log p^{(i,j)} + (1 - y^{(i,j)}) \log(1 - p^{(i,j)}) \right] + \text{regularization}.$$

Notes: Implicit negatives depend on what users had opportunity to see. A model trained on displayed-but-unclicked items learns behavior under the prior serving policy unless exposure is handled deliberately.

20 Content-Based Recommender Systems

Notation and Symbols

Symbol	Meaning in this chapter
$x_u^{(j)}$	Feature vector describing user j or user context.
$x_m^{(i)}$	Feature vector describing item i .
$v_u^{(j)}$	Learned user embedding produced by a user network.
$v_m^{(i)}$	Learned item embedding produced by an item network.
θ_u, θ_m	Learned parameters of the user and item networks.
$s(i, j)$	Predicted affinity between item i and user j .
λ	Regularization strength for network parameters.
k	Number of candidates retained for ranking or recommendation.

20.1 Two-Tower Representation Learning

User and item embeddings

With user descriptors $x_u^{(j)}$ and item descriptors $x_m^{(i)}$, learn

$$v_u^{(j)} = f_u(x_u^{(j)}; \theta_u) \in \mathbb{R}^k, \quad v_m^{(i)} = f_m(x_m^{(i)}; \theta_m) \in \mathbb{R}^k,$$

$$s(i, j) = (v_u^{(j)})^T v_m^{(i)}.$$

Derivation: Although the two networks accept different raw information, their final coordinates share a comparison space. High dot product is trained to mean high compatibility.

Use: Represent new items from their content before a large interaction history exists, reducing a key collaborative-filtering cold-start failure.

Squared rating objective with explicit network regularization

For explicit ratings, fit the two networks only on observed interactions and penalize their learned parameters:

$$J = \sum_{(i,j):r(i,j)=1} \frac{1}{2} \left((v_u^{(j)})^T v_m^{(i)} - y^{(i,j)} \right)^2 + \frac{\lambda}{2} (\|\theta_u\|_F^2 + \|\theta_m\|_F^2).$$

Derivation: The prediction error couples the two embedding networks; the penalty limits parameter growth and implements the regularization choice discussed in model diagnosis. For binary engagement, replace squared error with sigmoid cross-entropy under an explicit exposure or negative-sampling convention.

Unit-normalized embeddings

Normalize the final user and item vectors before comparing them:

$$\bar{v}_u^{(j)} = \frac{v_u^{(j)}}{\|v_u^{(j)}\|_2}, \quad \bar{v}_m^{(i)} = \frac{v_m^{(i)}}{\|v_m^{(i)}\|_2}, \quad s(i, j) = (\bar{v}_u^{(j)})^T \bar{v}_m^{(i)}.$$

For unit vectors,

$$\left\| \bar{v}_u^{(j)} - \bar{v}_m^{(i)} \right\|_2^2 = 2 - 2s(i, j).$$

Derivation: Normalization prevents embedding length alone from inflating affinity; ranking by largest dot product then matches ranking by smallest squared distance.

20.2 Retrieval, Ranking, and Computational Scale

Two-stage recommendation

For a very large item set, first retrieve a candidate collection $\mathcal{C}(j)$ using fast embedding similarity, then evaluate a richer ranker:

$$\mathcal{C}(j) = \text{TopK}_i \ s_{\text{retrieve}}(i, j), \quad \text{Rank}_j(i) = s_{\text{rank}}(i, j), \quad i \in \mathcal{C}(j).$$

Cue	Reading
Relevant item never enters candidates	Ranking improvements cannot display it; improve retrieval coverage.
Candidate set good but top order poor	Improve ranking features/objective or downstream constraints.
New catalog items lack exposure	Content embeddings and controlled exploration are important.
Scoring every item is too expensive	Use approximate retrieval followed by a more expressive candidate ranker.

20.3 Offline Metrics and Feedback Loops

Observed-data feedback

If a model predominantly exposes items it already favors, later clicks largely come from those items. Training then confounds preference with exposure:

$$\Pr(\text{interaction}) = \Pr(\text{exposed}) \Pr(\text{interaction} \mid \text{exposed}).$$

Cue	Reading
Clicks rise while long-term value falls	The optimized proxy is incomplete; revise metric or constraints.
Popular items dominate impressions	Preserve discovery/exploration and measure new-item coverage.
Unsafe or low-quality material scores highly	Add eligibility and safety constraints before ranking optimization.
Offline ranking improves under historical exposure	Validate online with carefully monitored experiments.

Notes: Recommendation is a decision system that changes future training data. Utility, diversity, safety, and exposure are not automatically encoded by click prediction.

21 Principal Component Analysis

Notation and Symbols

Symbol	Meaning in this chapter
$x^{(i)} \in \mathbb{R}^n$	Mean-normalized example in the original feature space.
$k < n$	Number of retained principal components.
$U_{\text{reduce}} \in \mathbb{R}^{n \times k}$	Matrix whose columns are retained principal directions.
$z^{(i)} \in \mathbb{R}^k$	Compressed coordinates of example i .
$\hat{x}^{(i)}$	Approximate reconstruction from compressed coordinates.
Σ	Covariance-like matrix of normalized input features.

21.1 Dimensionality Reduction Without Targets

Purpose

Principal component analysis reduces many measured features to a smaller set of coordinates while preserving directions of high variation in the inputs. It has no target y :

$$x^{(i)} \mapsto z^{(i)} \in \mathbb{R}^k, \quad k < n.$$

Use: Compress data, visualize high-dimensional examples in two or three coordinates, or produce a lower-dimensional representation when that serves an analysis goal.

Notes: PCA is not linear regression. Regression predicts a designated y by measuring vertical prediction error; PCA represents variation in the x 's without a supervised target.

21.2 Preprocessing and Directions

Mean normalization and scaling

Center each feature before learning directions:

$$x_j^{(i)} \leftarrow x_j^{(i)} - \mu_j, \quad \mu_j = \frac{1}{m} \sum_{i=1}^m x_j^{(i)}.$$

If features have widely different scales, scale them as well so the largest numerical unit does not determine the direction of maximum variation.

Principal directions

Using normalized examples, form

$$\Sigma = \frac{1}{m} \sum_{i=1}^m x^{(i)} (x^{(i)})^T.$$

The first principal direction is a unit vector capturing the largest projected variance; successive orthogonal directions capture the next largest remaining variance. Stack the selected directions in U_{reduce} . *Derivation:* For a unit direction u , projected coordinates are $u^T x^{(i)}$. Maximizing their squared magnitude selects a line along which the cloud of centered data varies most; eigenvectors of Σ solve this constrained maximization.

21.3 Projection and Reconstruction

Compressing an example

$$z^{(i)} = U_{\text{reduce}}^T x^{(i)}.$$

Approximate reconstruction

$$\hat{x}^{(i)} = U_{\text{reduce}} z^{(i)} = U_{\text{reduce}} U_{\text{reduce}}^T x^{(i)}.$$

Use: Interpret what is kept and lost when replacing the original features with compressed coordinates.

Notes: The fraction of explained variance indicates compression loss and helps choose k ; low reconstruction loss does not automatically improve a downstream prediction task. Validate any predictive pipeline end to end.

21.4 Appropriate and Inappropriate Uses

Cue	Reading
Visualization or compression	PCA can give compact coordinates with quantified retained variation.
Supervised prediction problem	Do not use PCA merely as a default; check whether it improves validation results.
Overfitting diagnosis	Regularization or additional data addresses variance directly; PCA is not a universal fix.
Interpretability requirement	Principal components mix original variables and may be harder to explain.

21.5 Principal Directions as a Variance Problem

Centered data matrix

Given examples as rows, subtract the training mean:

$$X_c = \begin{bmatrix} (x^{(1)} - \mu)^T \\ \vdots \\ (x^{(m)} - \mu)^T \end{bmatrix}, \quad \mu = \frac{1}{m} \sum_{i=1}^m x^{(i)}.$$

The covariance estimate is

$$\Sigma = \frac{1}{m} X_c^T X_c.$$

First principal component

For a unit direction u , projected coordinate is $z^{(i)} = u^T(x^{(i)} - \mu)$. Choose

$$u_1 = \arg \max_{\|u\|_2=1} u^T \Sigma u.$$

Derivation: The Lagrangian $u^T \Sigma u - \lambda(u^T u - 1)$ yields $\Sigma u = \lambda u$. The maximizing direction is the eigenvector with largest eigenvalue, carrying the greatest centered variance.

21.6 Multiple Components and SVD

Projection matrix

Order eigenvectors by eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$, and let

$$U_k = [u_1 \ \dots \ u_k] \in \mathbb{R}^{n \times k}.$$

For centered observation $x - \mu$,

$$z = U_k^T (x - \mu) \in \mathbb{R}^k, \quad \hat{x} = \mu + U_k z.$$

Equivalently, singular value decomposition

$$X_c = U S V^T$$

uses the leading columns of V as principal feature directions. *Use:* Compress correlated inputs, visualize low-dimensional variation, or remove redundant directions when information loss is acceptable.

Reconstruction error and explained variance

$$\frac{\sum_{i=1}^m \|x^{(i)} - \hat{x}^{(i)}\|_2^2}{\sum_{i=1}^m \|x^{(i)} - \mu\|_2^2} = 1 - \frac{\sum_{r=1}^k \lambda_r}{\sum_{r=1}^n \lambda_r}.$$

Derivation: PCA preserves variance along retained orthogonal directions and loses variance along omitted ones. Selecting k by a retained-variance criterion is selecting a compression tolerance, not predictive accuracy.

21.7 Scaling and Leakage

Unit dependence

If feature units differ materially, standardize before PCA when equalized variation is desired:

$$z_j = \frac{x_j - \mu_j}{s_j}.$$

The means, scales, and principal directions must all be fitted only from training data before applying to validation or test examples.

Cue	Reading
Need two-dimensional exploratory visualization	PCA can summarize major variance but may hide task-relevant rare structure.
Need prediction of labelled outcome	Select k using downstream validation, not variance retention alone.
Want to prevent overfitting automatically	PCA is not a guaranteed cure; discarding low variance can discard predictive signal.
Features have incomparable units	Scaling choice changes which directions count as large variance.

21.8 PCA Is Not Feature Selection

Rotated components

A component generally blends original variables:

$$z_r = u_r^T (x - \mu) = \sum_{j=1}^n u_{jr} (x_j - \mu_j).$$

Notes: Compression can reduce storage or computation, but transformed dimensions may be less interpretable than original variables. Use it when the tradeoff is appropriate to the downstream decision.

22 Reinforcement Learning Foundations

Notation and Symbols

Symbol	Meaning in this chapter
s	Current state observed by the agent.
a	Action selected in a state.
$R(s)$	Immediate reward received in a state or transition.
$\gamma \in [0, 1)$	Discount factor valuing delayed rewards.
$\pi(a s)$	Policy specifying action choices.
G_t	Discounted return after time t .
$V^\pi(s)$	Expected return from state s under policy π .

22.1 Sequential Decisions

Return

Unlike supervised learning, an action may alter future observations and rewards. A discounted return from time t is

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

Derivation: The discount factor expresses how future reward is valued and, for bounded rewards with $\gamma < 1$, makes an infinite sum finite.

Use: Compare actions whose benefits or harms arrive at different times.

Policy value

$$V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s].$$

A policy is good not because one immediate reward is high, but because the full expected sequence of resulting rewards is high.

22.2 Exploration and Exploitation

ϵ -greedy action selection

Given current action-value estimates $Q(s, a)$,

$$a = \begin{cases} \text{a random action,} & \text{with probability } \epsilon, \\ \arg \max_{a'} Q(s, a'), & \text{with probability } 1 - \epsilon. \end{cases}$$

Use: Gather evidence about actions not currently believed best while usually exploiting present knowledge.

Notes: Pure exploitation can lock a system into a poor action because alternatives were never adequately tried. Pure exploration fails to capitalize on useful knowledge.

22.3 Rewards Encode the Goal

Cue	Reading
Immediate proxy reward only	Agent can exploit the proxy while damaging long-term outcomes.
Large delayed consequence	Discount and horizon choices materially affect learned policy.
Unsafe action possible	Hard restrictions or safe exploration may be required beyond reward penalties.
Historical logs only	Action-value estimates inherit the behavior policy's coverage limitations.

Notes: Reward design is part of problem definition. A correctly optimized mistaken reward is still a mistaken system.

22.4 Markov Decision Process Notation

Dynamics and reward

A Markov decision process contains state space $\mathcal{S}$, action space $\mathcal{A}$, transition probabilities and reward:

$$P(s' | s, a), \quad R(s, a, s').$$

The Markov assumption states that, given current s_t, a_t , the distribution of next outcomes does not require the full earlier history:

$$\Pr(s_{t+1}, r_{t+1} | s_0, a_0, \dots, s_t, a_t) = \Pr(s_{t+1}, r_{t+1} | s_t, a_t).$$

Use: Define what information a state representation must preserve for sequential decisions. If necessary history is omitted, the learned policy faces a partially observed problem.

22.5 Discounted Return

Finite and infinite horizons

Starting at time t ,

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}, \quad 0 \leq \gamma < 1.$$

Derivation: Discounting can express preference for earlier reward and bounds an infinite sum when rewards are bounded. A smaller γ makes distant consequences exert weaker influence on current action selection.

Cue	Reading
Immediate reward omits later service failure	A policy may optimize the supplied reward while harming operations.
Consequences are substantially delayed	Choose horizon/discount and state representation so those consequences influence learning.
Safety event must almost never occur	Treat it as a constraint or decisive penalty, not a minor average reward adjustment.
Reward rises but real objective falls	Diagnose reward specification before assuming algorithm failure.

22.6 Policies and Value Functions

Policy

A stochastic policy specifies

$$\pi(a | s) = \Pr(A_t = a | S_t = s).$$

Its state value and action value are

$$V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s], \quad Q^\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a].$$

Policy value relation

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a | s) Q^\pi(s, a).$$

Derivation: At state s , the policy randomizes over the first action; its expected return is therefore the policy-weighted average of available action values.

22.7 Exploration

ϵ -greedy behavior

For current action-value estimate Q ,

$$A_t = \begin{cases} \text{random action,} & \text{with probability } \epsilon, \\ \arg \max_a Q(S_t, a), & \text{with probability } 1 - \epsilon. \end{cases}$$

Derivation: Pure exploitation never collects evidence about actions currently underestimated. Exploration creates data that can correct an early mistaken ranking of actions.

Use: Balance information gathering with current value; reduce exploration only when continued experimentation is less valuable or unsafe under the application constraints.

Notes: In high-stakes operation, random exploration may be unacceptable; safer experimental design, simulation, conservative policies, or human oversight may be required.

22.8 Continuous State Spaces

Why tabular storage stops working

In a simplified problem, a state may be one of finitely many positions. In a control problem, state can include real-valued position, velocity, angle, and angular velocity:

$$s = (x, y, \dot{x}, \dot{y}, \theta, \dot{\theta}, \dots) \in \mathbb{R}^d.$$

There are too many possible continuous states to store a reliable table entry for every $Q(s, a)$. Instead approximate action values with a function:

$$Q(s, a) \approx Q(s, a; w).$$

Derivation: Function approximation shares learned behavior between related states, allowing observed transitions to inform actions beyond one exact stored state.

Use: Motivate why the later Deep Q Network replaces a finite Q table in control applications such as landing or driving.

22.9 Stochastic Consequences

Expected next value

When an action can lead to multiple next states,

$$Q^\pi(s, a) = \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V^\pi(s')].$$

Use: Distinguish a good policy in expectation from a guaranteed outcome on every trajectory. Reliability and tail-risk requirements may require evaluation beyond average return.

23 Bellman Equations, Q-Learning, and Deep Q Networks

Notation and Symbols

Symbol	Meaning in this chapter
$Q^*(s, a)$	Optimal expected return after taking action a in state s .
s'	Next state reached after an action.
r	Immediate reward observed on a transition.
γ	Discount factor.
y_{target}	One-step Bellman training target.
$Q(s, a; w)$	Neural-network action-value approximation.
$\mathcal{B}$	Mini-batch sampled from replay memory.

23.1 Optimal Action Values

Bellman optimality equation

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right].$$

Derivation: An action's value equals its immediate reward plus the discounted value of behaving optimally after the next state is reached. This recursive identity links long-run value to one-step transitions.

Use: Define the target that value-based learning attempts to satisfy.

Optimal policy from action values

$$\pi^*(s) = \arg \max_a Q^*(s, a).$$

Notes: The maximization chooses actions using values; it does not remove the need to explore while values are being learned.

23.2 Tabular Q-Learning

Temporal-difference update

For an observed transition (s, a, r, s') ,

$$y_{\text{target}} = r + \gamma \max_{a'} Q(s', a'),$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha [y_{\text{target}} - Q(s, a)].$$

Derivation: The bracketed difference is a temporal-difference error: current value is corrected toward a one-step reward plus estimated best future value. Repeated representative transitions propagate distant rewards backward through earlier states.

Use: Learn action values when state-action pairs are small enough to store directly.

23.3 Deep Q Networks

Function approximation

For large state spaces, approximate action values using a neural network:

$$Q(s, a; w) \approx Q^*(s, a), \quad \mathcal{L}(w) = \frac{1}{|\mathcal{B}|} \sum_{(s, a, r, s') \in \mathcal{B}} (Q(s, a; w) - y_{\text{target}})^2.$$

For a terminal next state, use $y_{\text{target}} = r$; no future reward remains. *Derivation:* Each sampled transition becomes a supervised-like regression target, but the target itself depends on estimated future values. That moving dependency makes stability harder than ordinary labelled regression.

Replay memory and target network

$$y_{\text{target}} = r + \gamma \max_{a'} Q(s', a'; w^-),$$

$$w^- \leftarrow w \quad \text{only periodically or by slow averaging.}$$

Use: Replay batches reduce consecutive-transition correlation; a delayed target network slows target movement during optimization.

Notes: DQN joins neural prediction to sequential decision making. A low training temporal-difference loss is not alone proof of a valuable or safe policy; evaluate returns and constraints in the intended environment.

23.4 What Must Be Evaluated

Cue	Reading
Average episodic return	Measures declared reward accumulated under the policy.
Safety constraint violations	Detects unacceptable actions even if average reward improves.
Coverage across initial states	Prevents success from being confined to easy starts.
Shifted environment conditions	Tests whether policy remains useful when dynamics change.

23.5 Bellman Expectation and Optimality

Bellman expectation equation

For a fixed policy π ,

$$Q^\pi(s, a) = \mathbb{E} \left[r + \gamma \sum_{a'} \pi(a' | s') Q^\pi(s', a') \mid s, a \right].$$

Bellman optimality equation

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right].$$

Derivation: Policy evaluation averages future actions chosen by a specified policy. Optimality instead assumes the best available action will be selected after the transition. Confusing these equations changes which decision process is being valued.

Use: Recognize whether an algorithm is evaluating an existing policy or seeking a greedy improved policy.

23.6 Tabular Temporal-Difference Learning

One-step Q-learning target

For transition (s, a, r, s') ,

$$y = r + \gamma \max_{a'} Q(s', a'), \quad \delta = y - Q(s, a),$$
$$Q(s, a) \leftarrow Q(s, a) + \alpha \delta.$$

For terminal s' , set $y = r$. *Derivation:* The temporal-difference error δ compares current prediction with immediate reward plus bootstrapped future estimate. Repeated visits propagate information from rewarding outcomes to earlier choices.

Learning-rate implication

Large α reacts quickly but can chase stochastic transition noise; small α averages evidence more slowly. The state-action visitation pattern also matters: never-visited alternatives cannot be accurately valued.

23.7 Deep Q-Network Objective

Mini-batch Bellman regression

Approximate $Q(s, a; w)$ with a neural network and keep a delayed parameter set w^- for targets. For replay batch $\mathcal{B}$,

$$y_i = \begin{cases} r_i, & s'_i \text{ terminal,} \\ r_i + \gamma \max_{a'} Q(s'_i, a'; w^-), & \text{otherwise,} \end{cases}$$
$$\mathcal{L}(w) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} (Q(s_i, a_i; w) - y_i)^2.$$

Derivation: The update resembles supervised regression, but targets are generated from a changing value estimator. Freezing w^- temporarily reduces the moving target effect during several optimizer steps.

One inference pass for all discrete actions

For a state with A possible discrete actions, an efficient network maps

$$s \mapsto \begin{bmatrix} Q(s, a_1; w) \\ Q(s, a_2; w) \\ \vdots \\ Q(s, a_A; w) \end{bmatrix} \in \mathbb{R}^A.$$

Then both greedy action selection and Bellman maximization use one forward pass:

$$a^* = \arg \max_{a_r} Q(s, a_r; w), \quad \max_{a'} Q(s', a'; w^-).$$

Derivation: A network that accepts one state-action pair would need a separate evaluation for every available action. Outputting all action values at once shares computation across the action comparison needed at each state.

Use: Capture the efficient DQN architecture for finite action spaces presented in the reinforcement-learning material.

Replay memory

Store observed transitions and sample mini-batches rather than training only on the most recent consecutive trajectory:

$$\mathcal{D}_{\text{replay}} = \{(s_t, a_t, r_{t+1}, s_{t+1})\}_t.$$

Derivation: Consecutive states are highly correlated; random replay improves data mixing and reuses informative transitions. It does not make a biased behavior policy or incomplete reward specification disappear.

Cue	Reading
Training TD loss small but returns low	Targets can be self-consistent under poor coverage or reward; evaluate actual policies.
Learning becomes unstable	Inspect learning rate, reward scale, replay diversity, and target-network update rate.
Policy never attempts valuable alternative	Improve safe exploration or data coverage before judging values.
Offline transitions come from another policy	Recognize coverage limits; actions absent from data cannot be reliably evaluated.

23.8 Target-Network Updates

Hard and soft updates

A hard update copies parameters every C steps:

$$w^- \leftarrow w.$$

A soft update slowly tracks current parameters:

$$w^- \leftarrow \tau w + (1 - \tau)w^-, \quad 0 < \tau \ll 1.$$

Use: Control how rapidly bootstrapped targets move while online parameters are being optimized.

Notes: Stability mechanisms solve an optimization difficulty of function approximation with bootstrapping; they do not certify safe reward definitions, representative environments, or acceptable policies.

23.9 Policy Evaluation Before Deployment

Measure return and constraints over episodes

For evaluation episodes $e = 1, \dots, N$,

$$\hat{V}(\pi) = \frac{1}{N} \sum_{e=1}^N G_0^{(e)}, \quad \hat{p}_{\text{violation}} = \frac{1}{N} \sum_{e=1}^N \mathbf{1}\{\text{constraint violation in episode } e\}.$$

Cue	Reading
Average return improves but violation rate rises	Reject or constrain the policy under safety requirements.
Performance measured only from easy initial states	Expand evaluation coverage across states likely in operation.
Environment dynamics shift seasonally	Test representative shifts before trusting historical return.
Reward omits a delayed operational cost	Revise the objective; no Bellman update can optimize an unrepresented consequence.

24 Theory Checkpoint III

1. The smallest K-means distortion selects the correct segmentation

Statement. When comparing candidate values $K = 2, \dots, 30$ on one unlabeled dataset, choosing the K with the smallest K-means distortion is a principled standalone rule for discovering the most meaningful cluster count.

Answer: False.

Explanation. Distortion cannot increase as more centroids are permitted; in the extreme, enough centroids can place each observation very near its own center. The objective measures compactness for a fixed K , not usefulness of the number of groups. The choice requires an elbow judgment, stability evidence, or an external purpose such as how many differentiated actions can be supported.

2. A known rare positive population always favors anomaly detection

Statement. If failures are rare, anomaly detection is necessarily preferable to supervised classification even when thousands of representative labelled failure examples are available and future failures are expected to resemble them.

Answer: False.

Explanation. Rarity alone is not decisive. A supervised classifier can learn a repeatable boundary when positive examples adequately represent future positives. Density-based anomaly detection is especially attractive when known positives are scarce or novel anomaly types matter. The question is information about the positive class, not only its base rate.

3. Missing ratings are negative labels in collaborative filtering

Statement. In a collaborative-filtering cost, setting every unobserved rating to zero and including it as an ordinary squared-error target improves inference because the system learns that users disliked items they never rated.

Answer: False.

Explanation. An unobserved entry ordinarily means unknown, not disliked. The indicator $r(i, j)$ excludes absent ratings from explicit-rating loss because otherwise exposure and missingness are mislabelled as preference. For implicit-feedback systems, negatives must be defined or sampled with care because non-clicked unseen content carries a different meaning from an explicit dislike.

4. A low DQN training loss proves the learned policy is high-value

Statement. If a Deep Q Network reaches a low squared Bellman-target loss on its replay buffer, the greedy policy has necessarily achieved high long-run reward in the operating environment.

Answer: False.

Explanation. The loss measures consistency with targets generated from sampled experiences and estimated future values; its targets move and its replay coverage may omit important states or actions. A network can fit biased transitions or an imperfect reward while performing badly in deployment. Policy evaluation needs achieved returns, safety checks, state coverage, and tests under relevant environment conditions.

5. Recommendation clicks and business value

[Business Scenario]

Statement. An online marketplace replaces its content-based recommender with a system that increases click-through rate by repeatedly showing highly sensational low-margin items. Long-term repeat purchases and seller diversity fall, but because the optimized click metric improved on a randomized test, the recommender is unambiguously better for the business.

Answer: False.

Explanation. The experiment establishes an improvement in the chosen proxy, not in every business objective. Recommendation changes future exposure and customer behavior; a click metric can reward short-term attraction while damaging retention, margin, or catalog health. The platform must define a metric or constrained objective aligned with value, for example optimizing relevant engagement subject to margin, retention, and safety requirements.

6. Inventory policy and delayed rewards

[Business Scenario]

Statement. A distribution center trains a reinforcement-learning replenishment policy with reward equal only to immediate daily holding-cost reduction. The learned policy orders almost nothing before a seasonal demand surge and then incurs severe stockouts. The result is primarily evidence that Bellman updates were incorrectly implemented, because an optimal policy could not choose actions that hurt later service levels.

Answer: False.

Explanation. An agent optimizes the reward it is given. If stockout penalties and delayed service consequences are absent or too heavily discounted, reducing immediate inventory cost can be optimal under the misspecified objective even while harming real operations. The first diagnosis is reward and horizon design, followed by evaluation across seasonal conditions and service constraints; only then does implementation debugging become the leading explanation.